

Transaction Level Modeling with SystemC for System Level Design

Organization of the Tutorial

Introduction
to
SystemC

Introduction to
TLM

Conclusion

Modeling
Elements
&
Simulation
Kernel

TLM2
Interfaces
& Coding
Styles

Outline

(Part-I: SystemC)

- [Module 1 - Introduction](#)
- [Module 2 - Terminology](#)
- [Module 3 - Event and Process](#)
- [Module 4 - Simulator](#)
- [Module 5 - Example of SystemC Model](#)



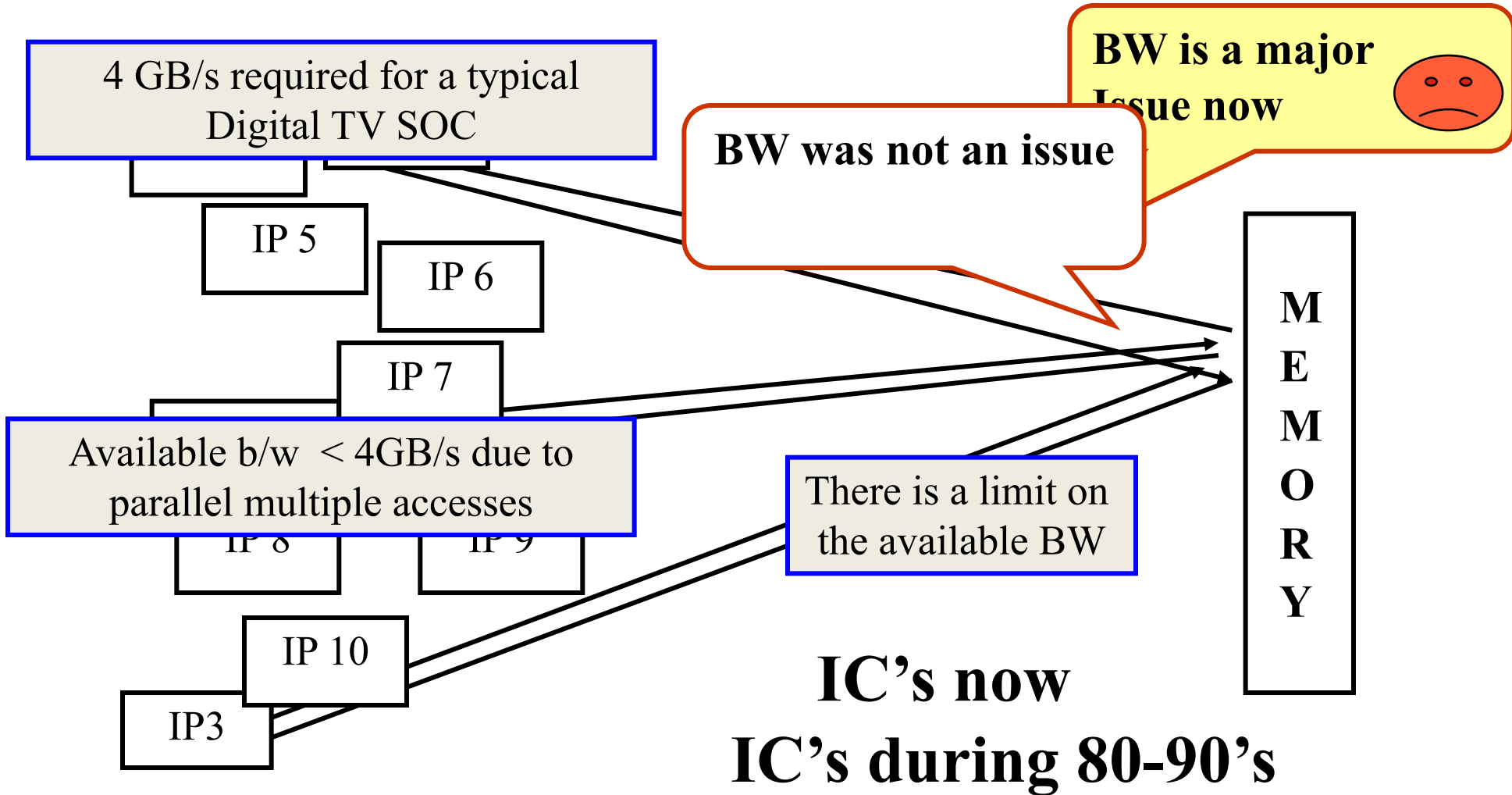
Module 1 - Introduction



- Inadequacy of C/C++ for System Level Design
- SystemC for System Level Design

Motivation to for introducing a new Modeling Language ?

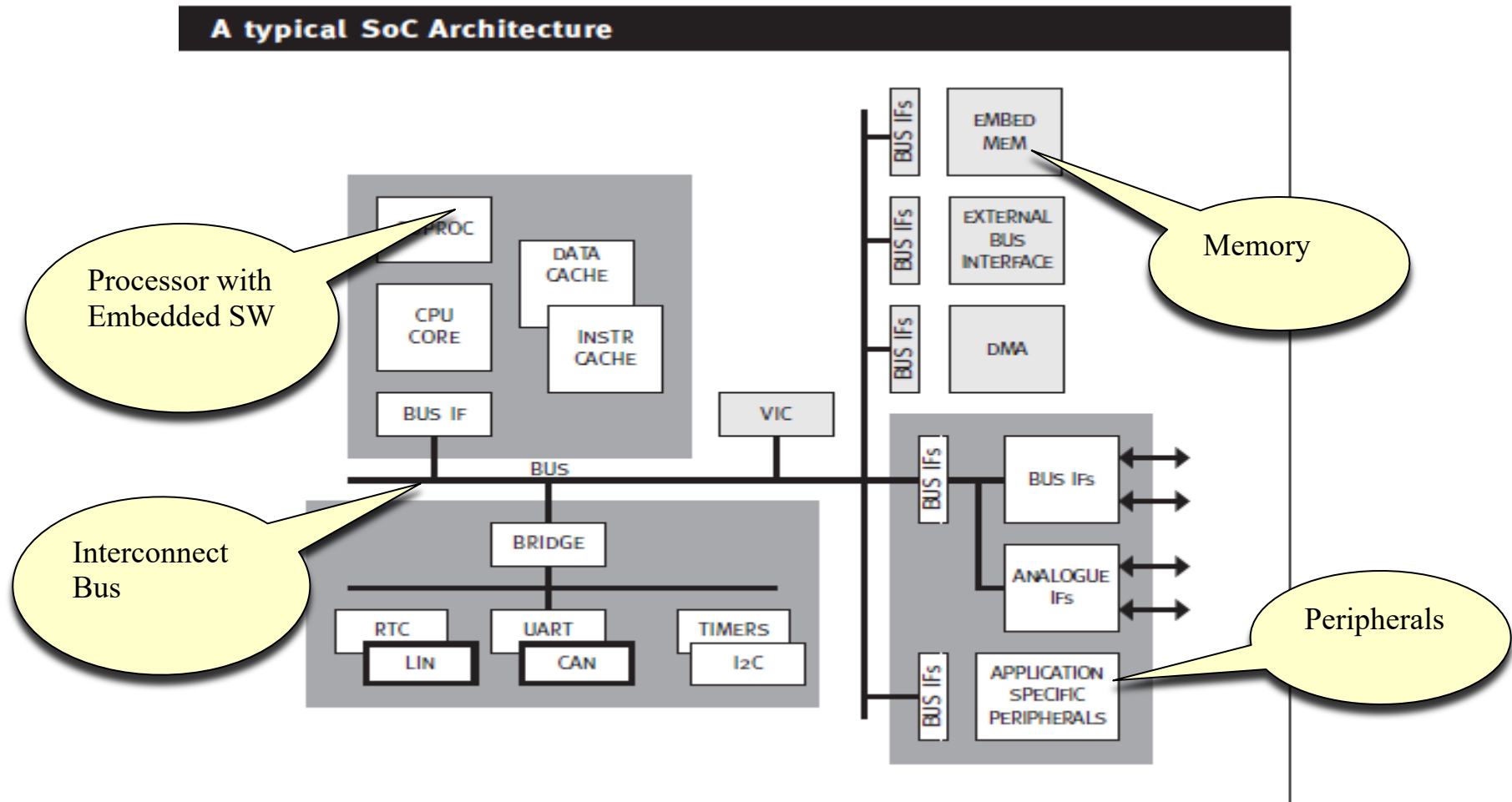
Why was it insufficient to keep using familiar HDLs ?



Motivation to for introducing a new Modeling Language ?

Why was it insufficient to keep using familiar HDLs ?

How does a system-on-chip like today?



Solutions & Limitations

1. Spreadsheet analysis

- Low accuracy when traffic from different IP's get mixed

2. High level C/C++ models

- Not available for complex SOC.
- Low accuracy
- Not effective correlation with post-silicon results

3. RTL Simulation with HDL- VHDL, Verilog

- Stable SOC RTL available very late.
- Very Slow though accurate

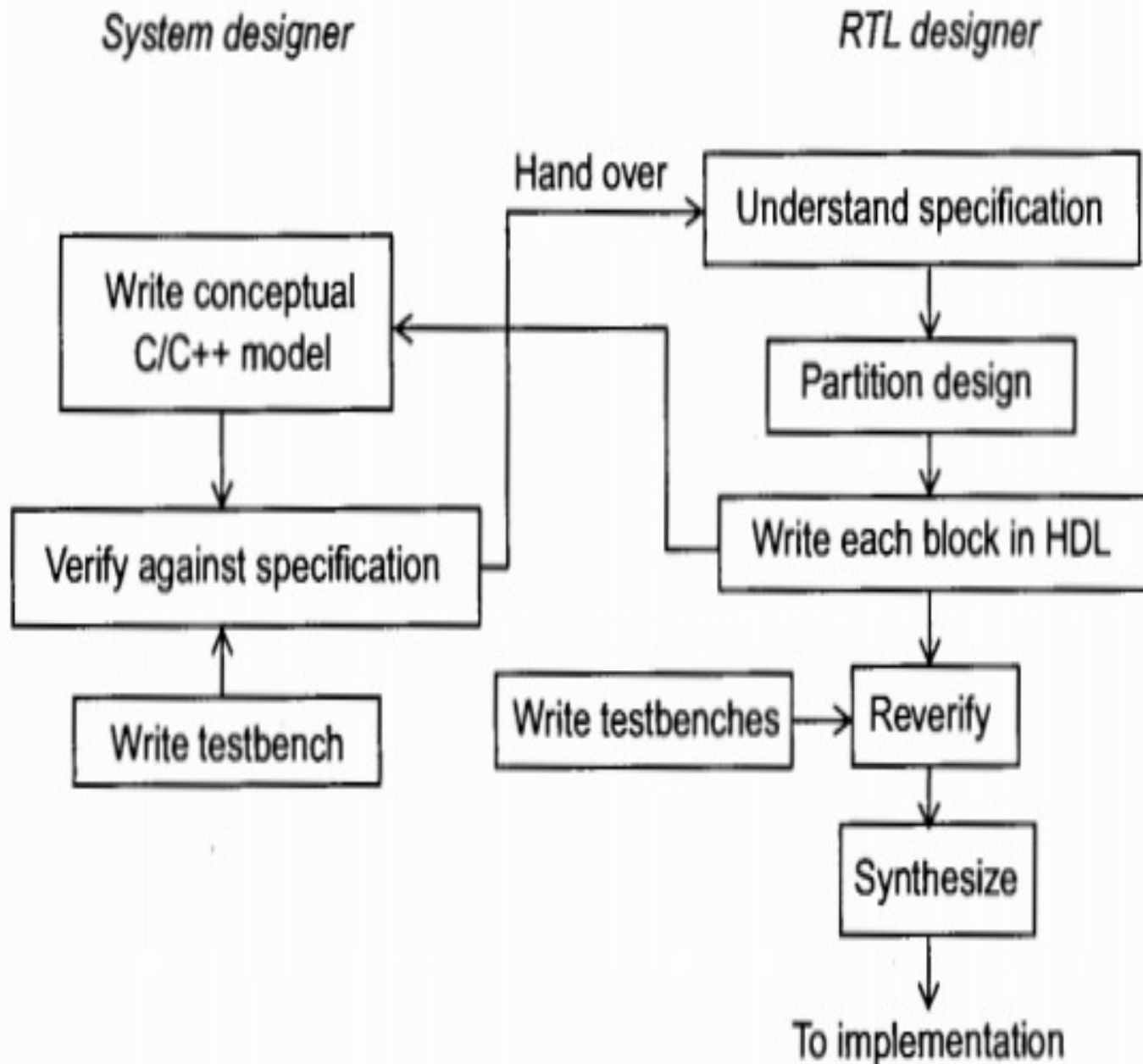
4. Emulation platforms

- Faster than RTL simulations but results are very late in SOC cycle
- RTL changes are very difficult without major impact on schedule

System Level Design

Inadequacy of C/C++ for System Level Design

Traditional C++ Based System Design Flow



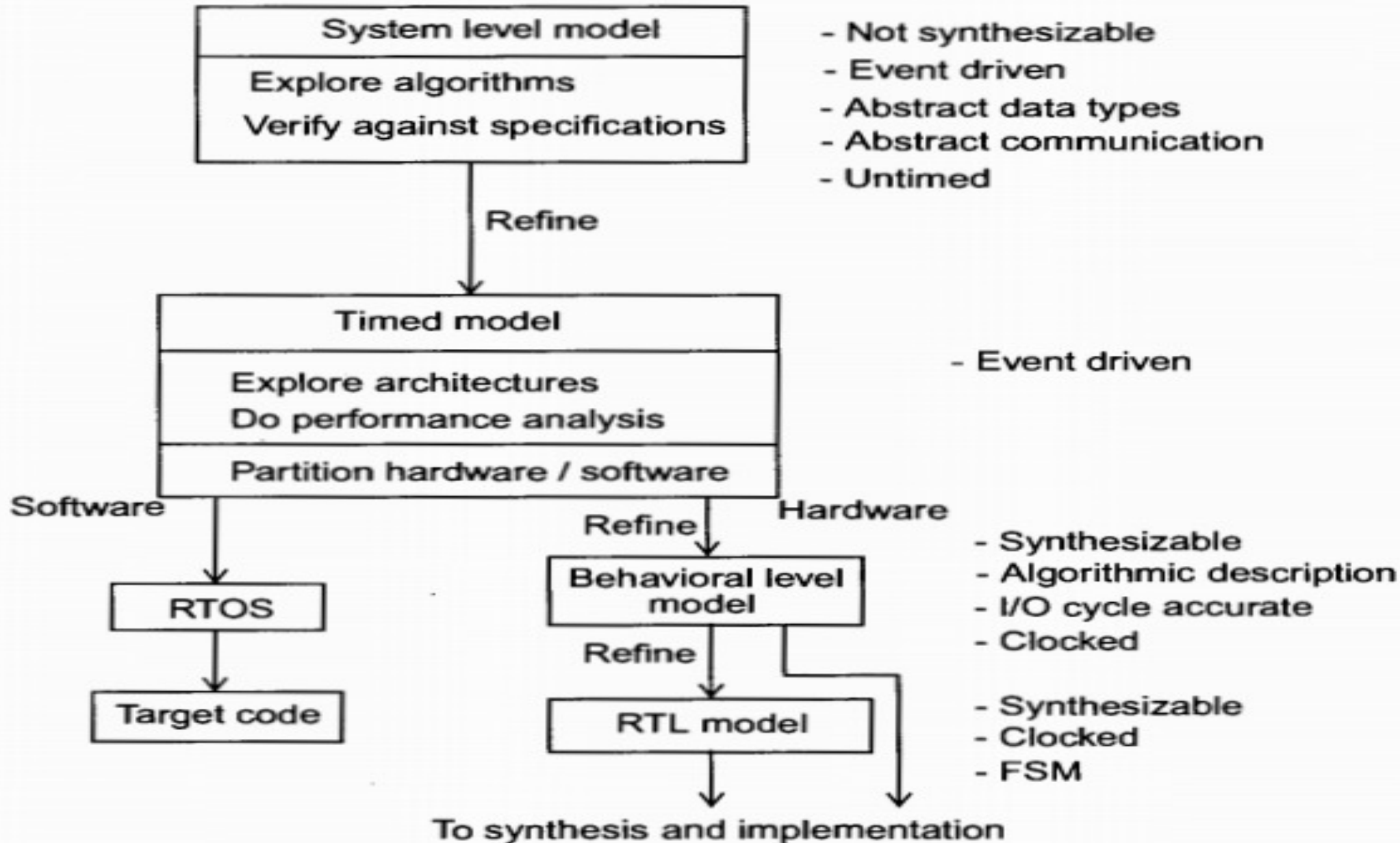
Advantages

- OOPS
- Existing Infrastructure

Disadvantages

- No Notion of Simulation Time
- No Concurrency Support
- No Availability of Hardware Data Types
- Manual Conversions creates errors
- Disconnect between System Model & HDL Model

Modern System Design Flow



Modeling Abstractions at System Level

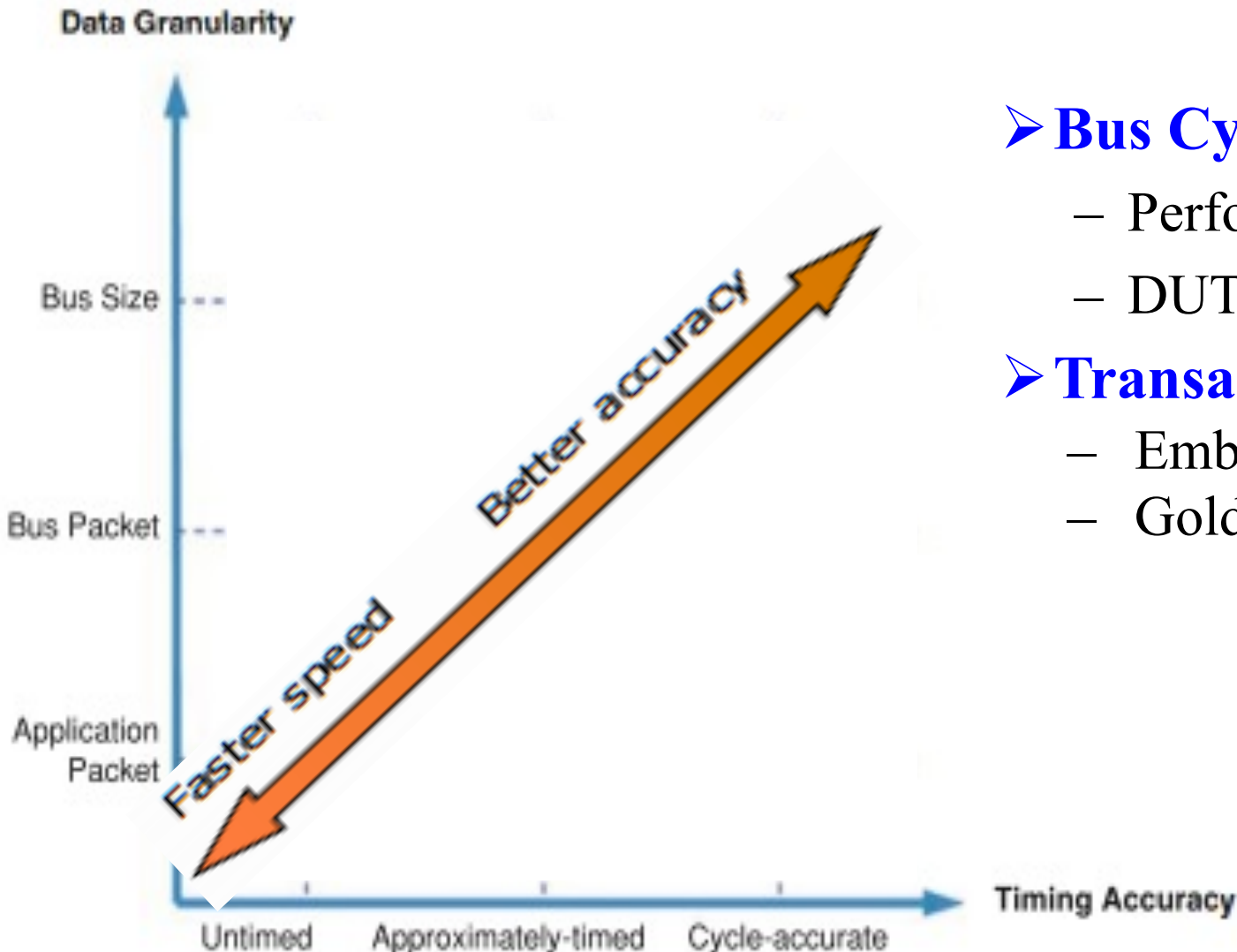
➤ **Register Transfer Accurate**

➤ **Bus Cycle Accurate**

- Performance Modeling
- DUT Verification

➤ **Transaction Level Accurate**

- Embedded S/w Development
- Golden Reference Models



Transaction Level Accuracy (TLM)

➤ TLM- Untimed (Programmers View- PV)

- Refers to both Interface & Functionality
 - Transport data in zero time
 - Process execute in zero time but in sequence
- Abstract Communication Channels/ Interfaces
 - Functional calls (i.e *read(addr), write(addr, data)*)

➤ TLM- Timed (Approximately or Loosely Timed)

- Timed but not clocked
- May refer to one or both Interface & Functionality
 - Transport data may be assigned latency
 - Processes may be assigned an execution time
- Abstract Communication Channels/ Interfaces
 - Functional Calls but latency modelled

Cycle Accuracy (CA)

➤ Bus Cycle Accurate

- Communication protocol may be clocked
- Communication Latency modelled
- Functionality may/not be timed

➤ Register Transfer Accurate

- Synchronous transfer between the functional units
- Fully timed/ Clocked
- Bit/ Register/ BUS Cycle Accurate
- May be synthesizable

Modeling Accuracy Requirement at System Level

➤ Modeling at Different Abstraction Level

- RTL, BCA, TLM

➤ Structural Accuracy

- Design Partitioning/ Modules/ Abstract Data Type

➤ Timing Accuracy

- Physical Time & Simulation Time/ Timed/ Clocked

➤ Functional Accuracy

- Timed/ Untimed/ Event Based/ Clock Based

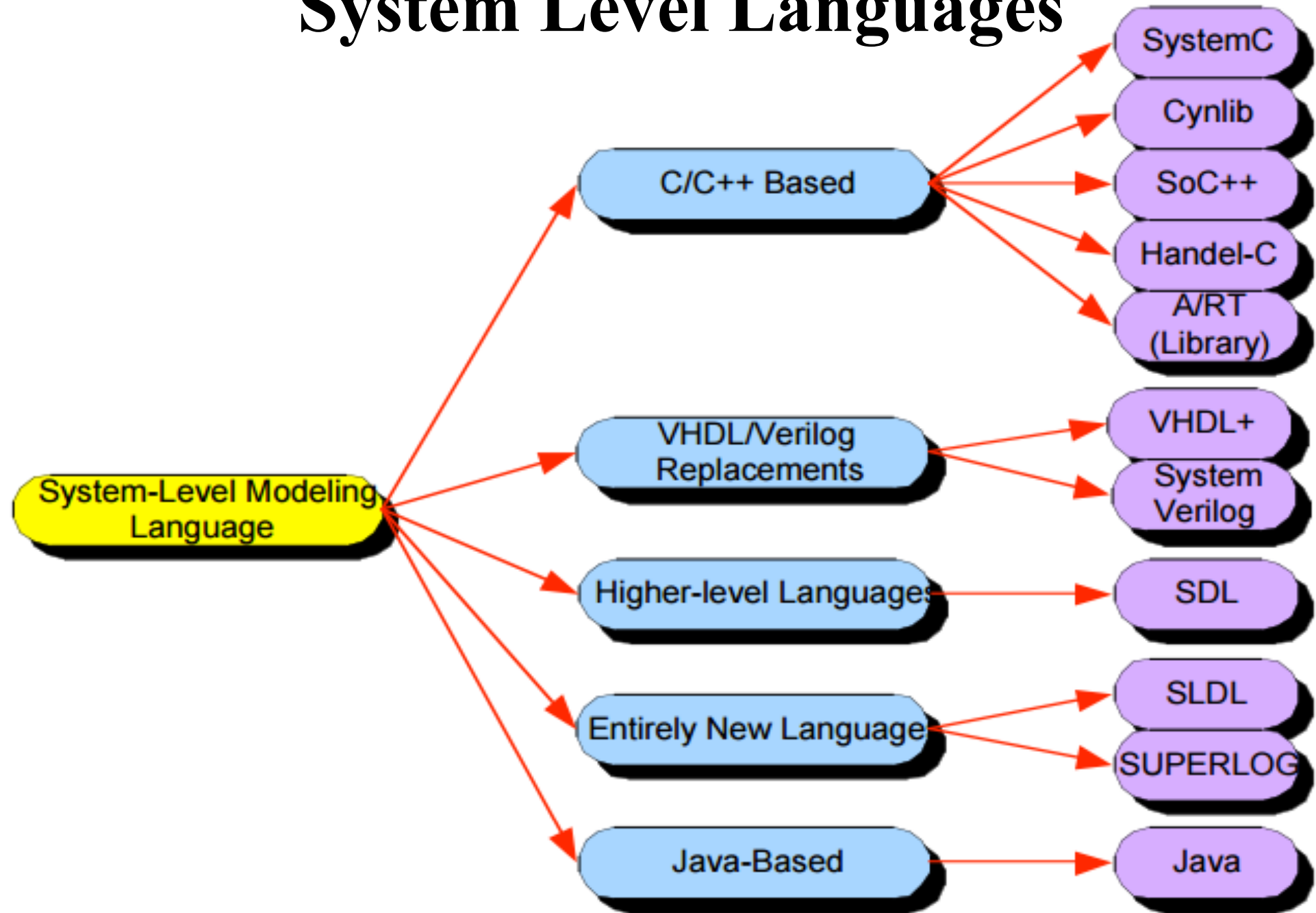
➤ Data Organization Accuracy

- Registers/ Hardware Data

➤ Communication Protocol Accuracy

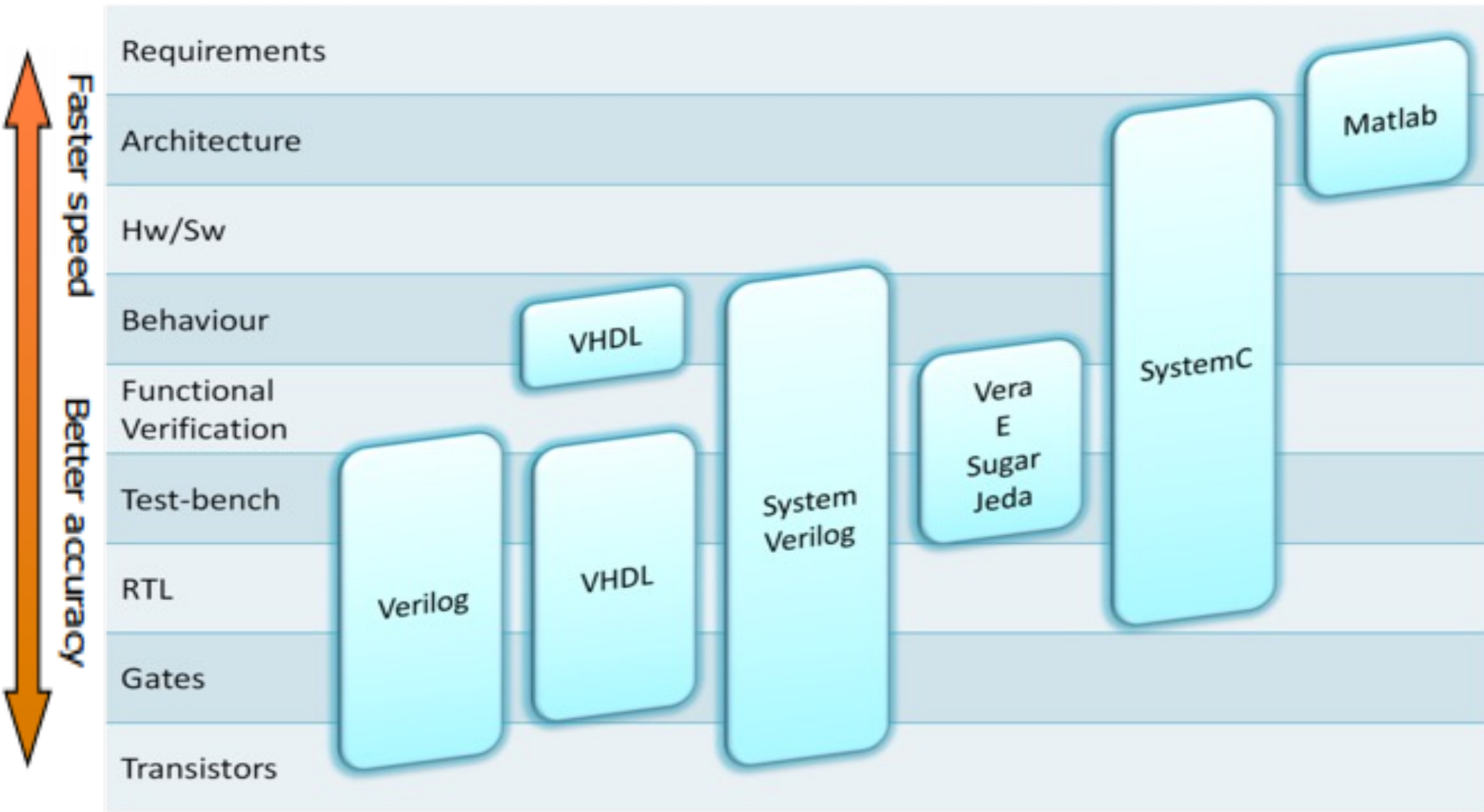
- Clocked/ Timed/ Signal Level Interface/ Functional Interface

System Level Languages



Abstraction Level Coverage of Various HDLs

Language Comparison

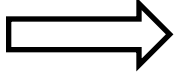
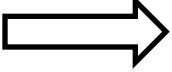
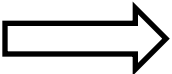
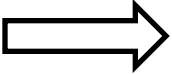


HDL- Comparison

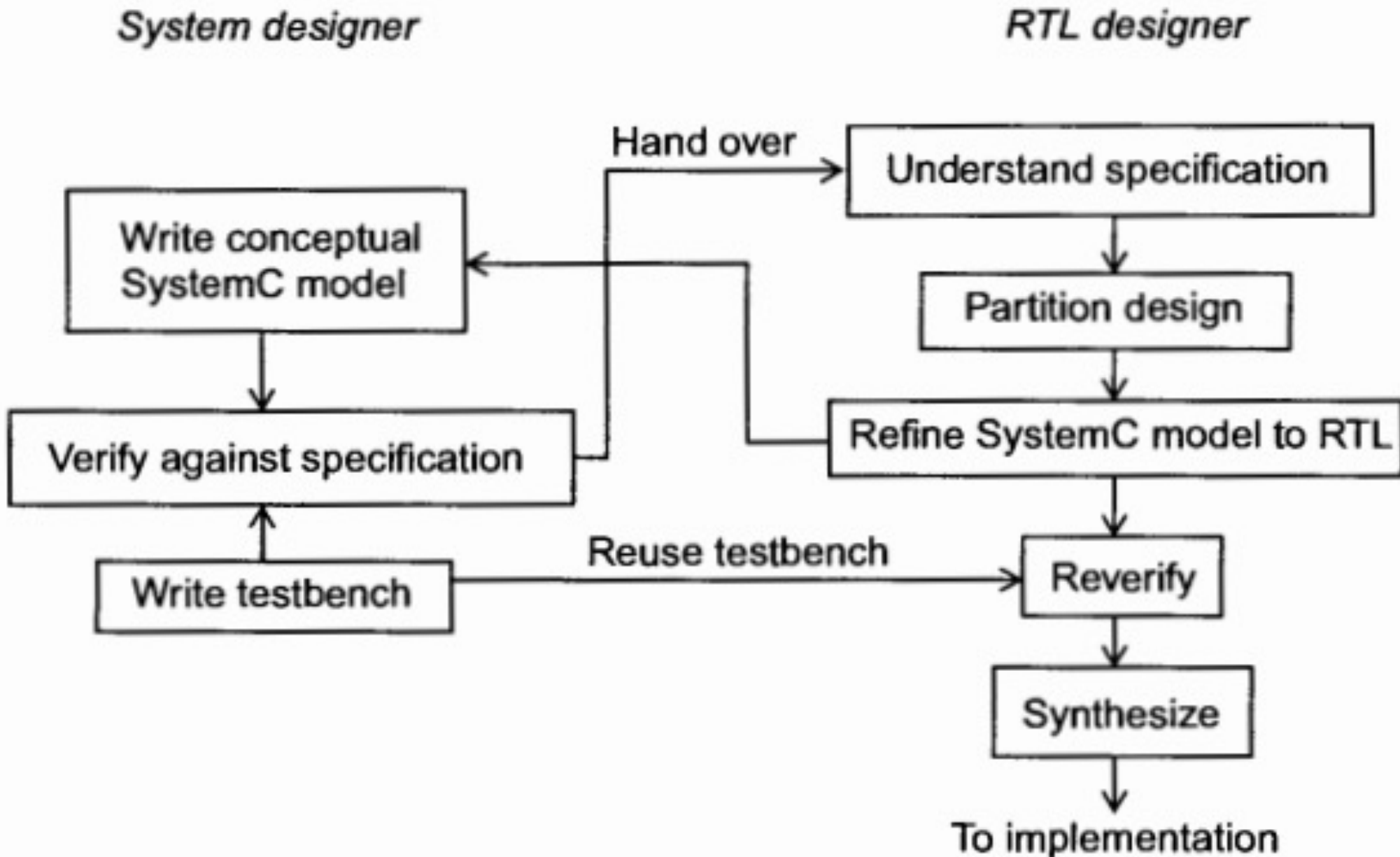
	VHDL	Verilog	SystemC	SystemVerilog
IEEE Standardization	1987	1995	2005	2005
Base Language	ADA/ Pascal	C	C++	Verilog/ C++
Type Checking	Strong	Weak	Strong	Medium
Design re-usability	Medium	Low	Very High	High
Ease of Learning	Difficult	Medium	Easy	Very Difficult
High Level Construct Support	High	Low	High	High
Low Level Construct Support	Medium	High	Low	High
Synthesizability	High	High	Low	High
Adoption	Very High	Very High	Medium	High ₁₇

SystemC for System Level Design

Reasons for Adopting **SystemC**

- **Single language for modeling hardware/software systems:**
 - Communication  channels, events
 - Concurrency  process
 - Timing  clock, sc_time
 - Data types  sc_logic
- **Object-oriented approach, design re-usability**
- **Hardware primitives along with simulation kernel**
- **Multiple abstraction levels**

Reasons for Adopting **SystemC**



Brief History & Major Releases

- **Confluence of 4 different streams of Ideas/ Efforts-**
 - **Synopsys & Univ. of California/ Infineon:** Cycle based approach deploying C++
 - **Frontier Design:** Data Types
 - **IMEC/ CoWare:** Hw/Sw Co-design
 - **OSCI:** System Level Design concepts

- **SystemC 1.0:** Production release on September 27th 1999
 - RTL-like concepts

- **SystemC 2.0:** Production release on October 18th 2001
 - System Level Modeling and Verification

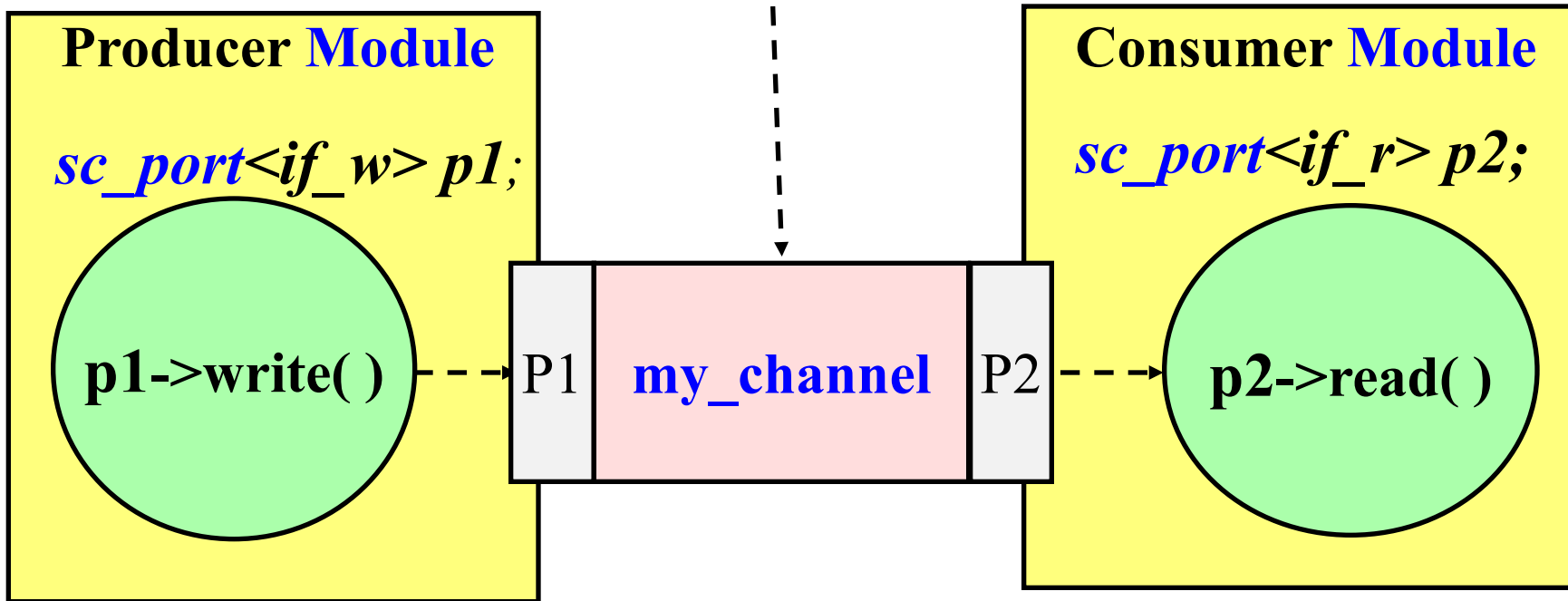
- **SystemC 2.1:** IEEE 1666-2005

- **SystemC 2.3.1:** IEEE 1666.1–2016 standard for SystemC AMS
 - Strong Process Control

Module 2- SystemC Terminology

SystemC is: **Module, Process, Ports, Channel, Interface**

```
class my_channel: if_w, if_r, sc_module {  
    void write( ) {...}  
    void read ( ) {...}  
};
```



```
class if_w: virtual sc_interface {  
    virtual void write() = 0;  
};
```

```
class if_r: virtual sc_interface {  
    virtual void read() = 0;  
};
```

Data Type

SystemC Data Types

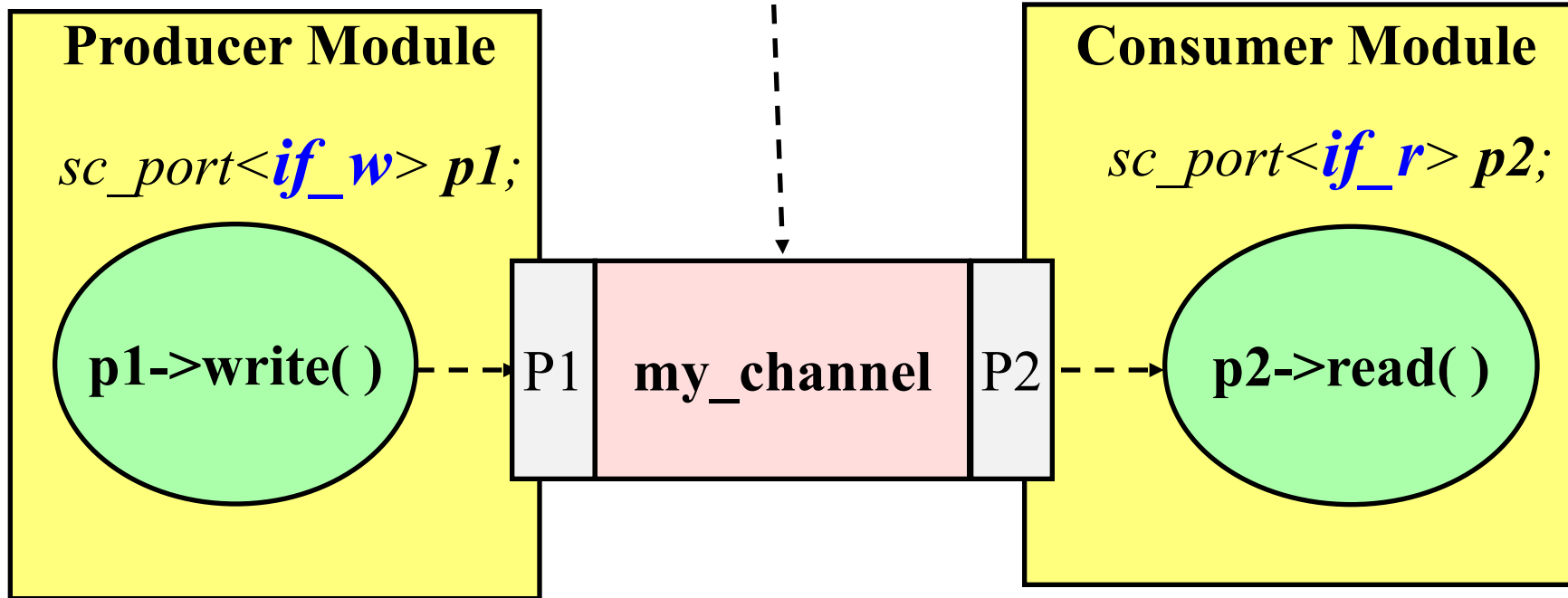
- All C++ data types
- **sc_logic** - 0, 1, X, Z
 - **sc_lv<length>** - array of sc_logic
- **sc_uint<length>** , **sc_biguint<length>**, **sc_bv<length>**
- **sc_fix**

Data Type

- Use native C++ data types wherever possible
- Use **sc_logic** if 4-valued logic is required
- Use **sc_uint<length>** for less than 64-bits and 2-valued logic
- Use **sc_biguint<length>** for more than 64-bits and 2-valued logic
- Use **sc_fix** for fixed point arithmetic

Interface

```
class my_channel: if_w, if_r, sc_module
{
    void write( ) {...}
    void read( ) {...}
};
```



```
class if_w: virtual sc_interface {
    virtual void write() = 0;
};
```

```
class if_r: virtual sc_interface {
    virtual void read() = 0;
};
```

SystemC Interface

- Declares a set of access methods needed for communication between IPs
- Do not provide any implementation of the access methods
- Implementation of interfaces are done within a **channel**
 - **sc_signal_in_if** → T& read() const interface
 - **sc_signal_inout_if** → void write(const T&) interface
- You can define your own interfaces but it should inherit from **sc_interface** class

```
template <typename T>
class my_interface : public sc_interface {
    int read(int & addr) = 0;
    void write( int& addr, int & data) = 0;
}
```

Interface examples

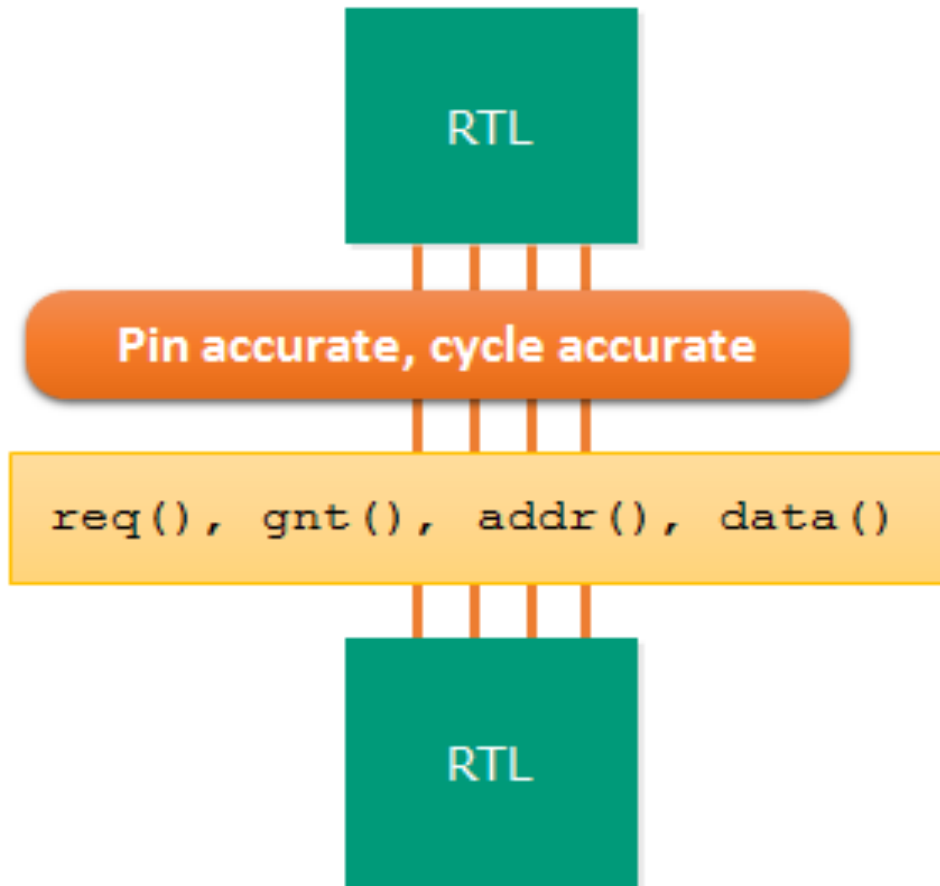
```
// -----  
// sc_signal_in_if  
// this interface provides a 'read' method  
// -----  
  
template <class T>  
class sc_signal_in_if : virtual public  
sc_interface {  
    public:  
        // interface methods  
        virtual const T& read() const = 0;  
};
```

```
// -----  
// sc_signal_inout_if  
// this interface provides 'read' & 'write' method  
// -----  
  
template <class T>  
Class sc_signal_inout_if : virtual public  
sc_signal_in_if {  
    public:  
        // interface methods  
        virtual void write( const T& ) = 0;  
};
```

Difference Interface Type for Different Abstraction Level

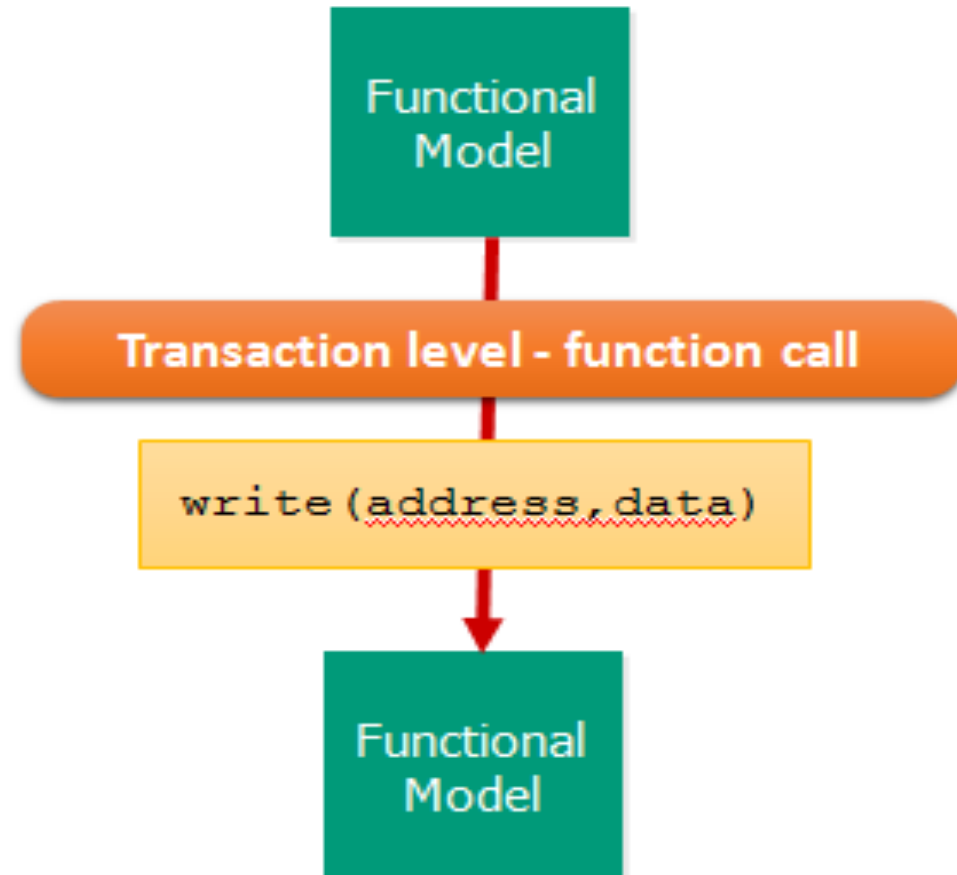
Cycle Accurate Model

Interfaces in form of signals of **request, grant, address, data**, etc



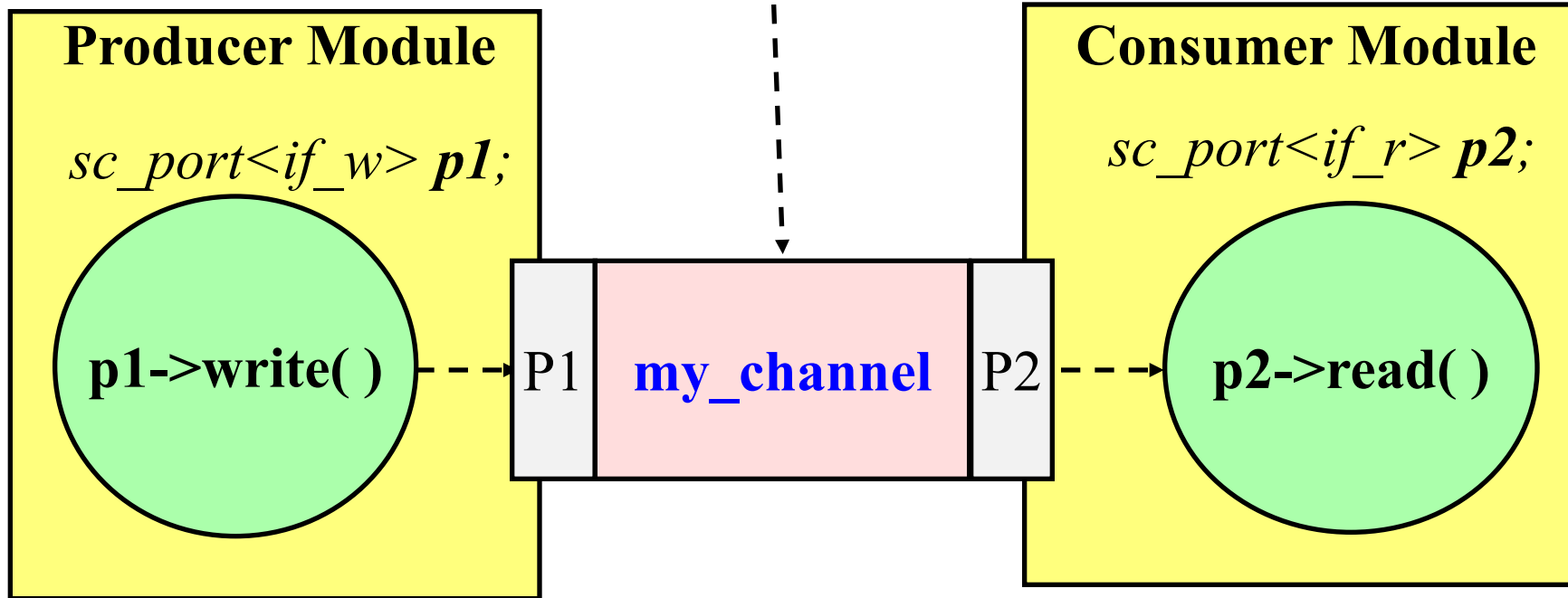
Transaction Level Model

Interfaces in the form of **read (address)** or **write (address, data)**



Channel

```
class my_channel: if_w, if_r, sc_module {  
    void write( ) {...}  
    void read( ) {...}  
};
```



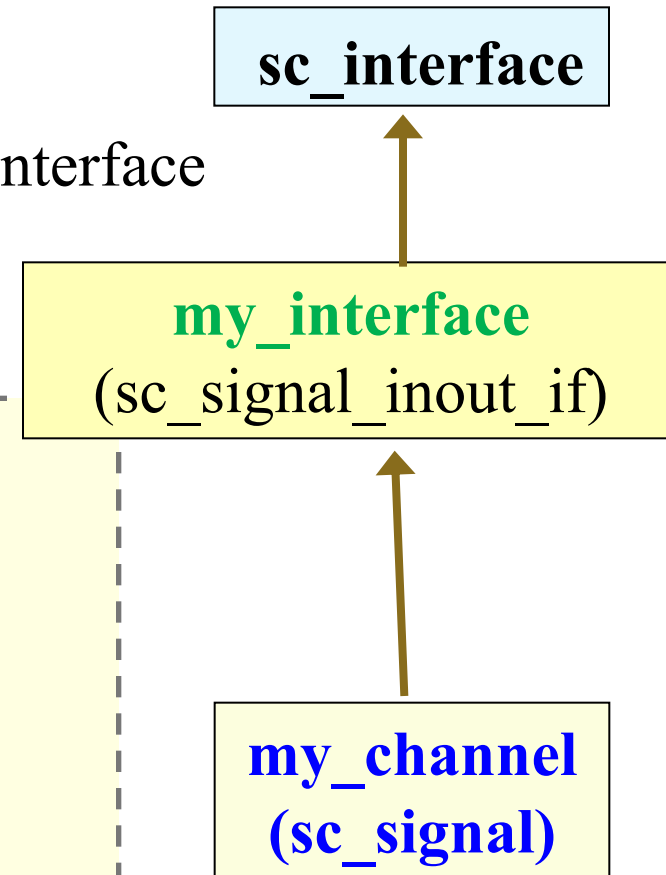
```
class if_w: virtual sc_interface {  
    virtual void write() = 0;  
};
```

```
class if_r: virtual sc_interface {  
    virtual void read() = 0;  
};
```

SystemC Channel?

- Means for communication between modules
- Provides the functionality of methods declared in interface
 - Derived from interfaces

```
template <typename T>
class my_channel : my_interface, sc_module {
    int read(int& addr) {
        return arr[addr];
    }
    void write(int & addr, int & data) {
        arr[addr]= data;
    }
}
```



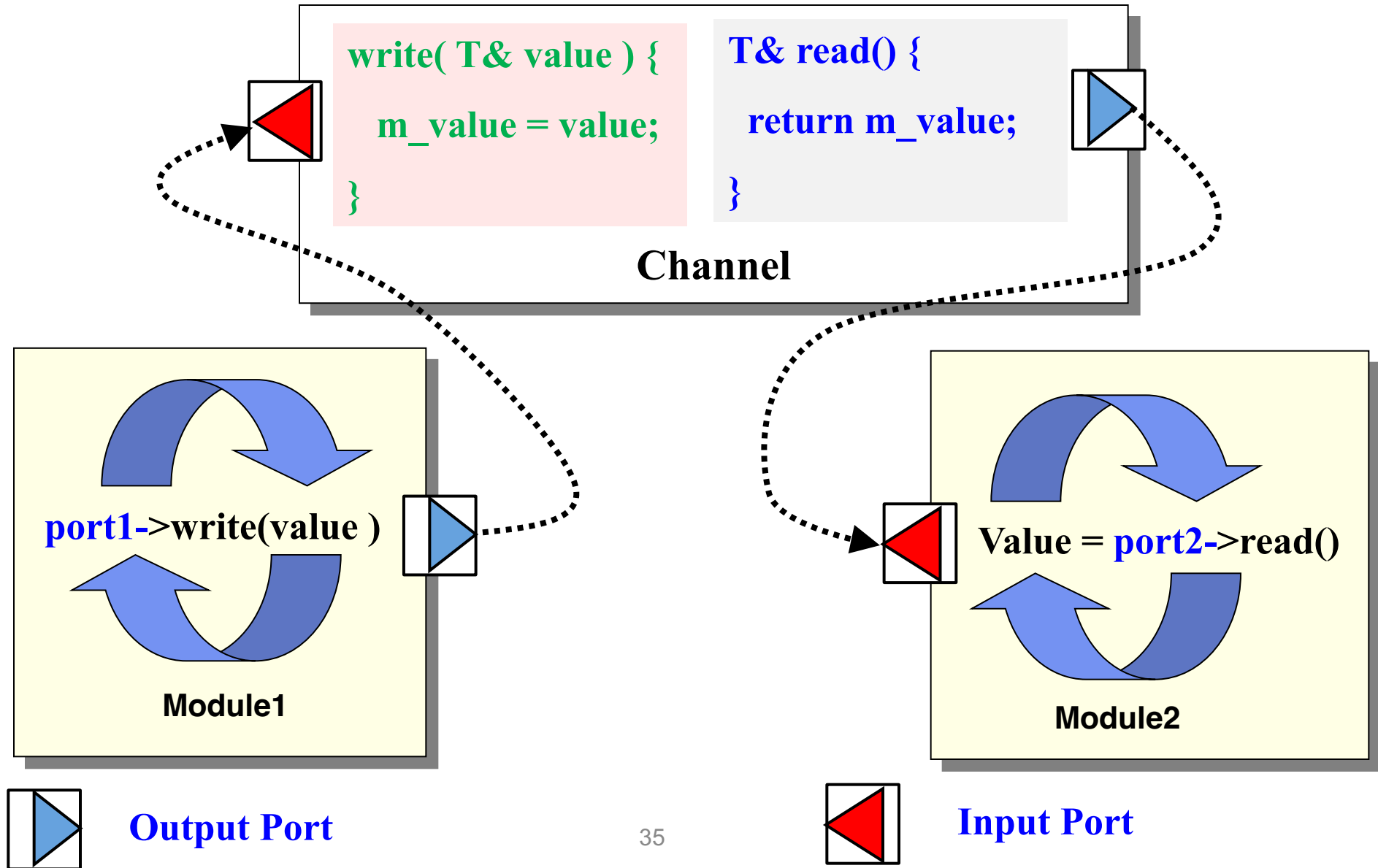
Channel Example (1/2)

```
// -----  
// sc_signal<T>  
// -----  
  
template <class T>  
class sc_signal : public sc_signal_inout_if<T> {  
    m_value;  
    T& read () {  
        //immediate return  
        return m_value;  
    }  
}
```

Channel Example (1/2)

```
// -----  
// sc_signal<T>  
// -----  
  
template <class T>  
class sc_signal : public sc_signal_inout_if<T> {  
    m_value;  
    T& read () {  
        //return after some time  
        wait (10, SC_NS);  
        return m_value;  
    }  
}
```

SystemC Port



SystemC Port

- Created from the base class **sc_port** and are bound to an interface type
- Syntax
sc_port<interface_type, N> port_name;
N is the number of channels connected to the port
- To access a channel method through an **sc_port** object, use **->** operator

```
sc_port<sc_signal_in_if<int>, 2> my_read_port;  
sc_port<sc_signal_out_if<int>, 2> my_write_port;  
int data = my_read_port->read(addr);  
my_write_port->write(addr, data);
```

SystemC Ports

// SystemC

```
sc_in  <signaltype> port1;  
sc_out <signaltype> port2 ;  
sc_inout<signaltype> port3;
```

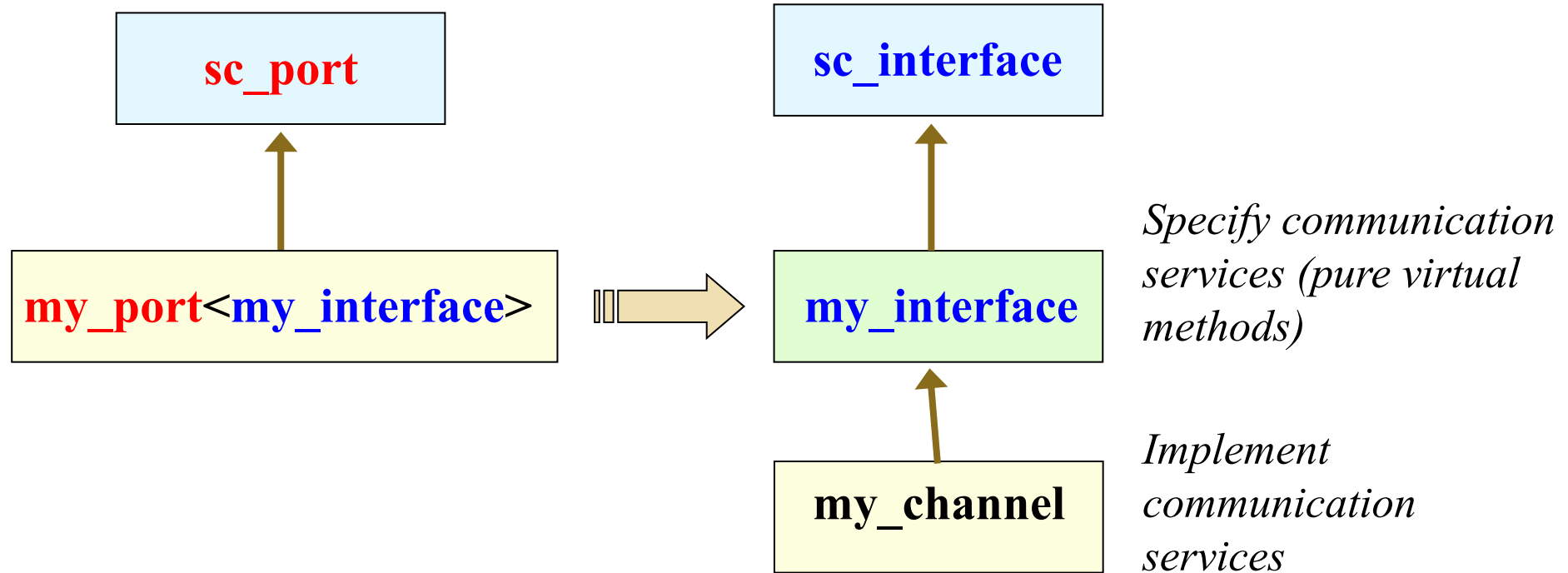
// Verilog

```
input  wire port1;  
output wire port2;  
inout  wire port3;
```

➤ Example

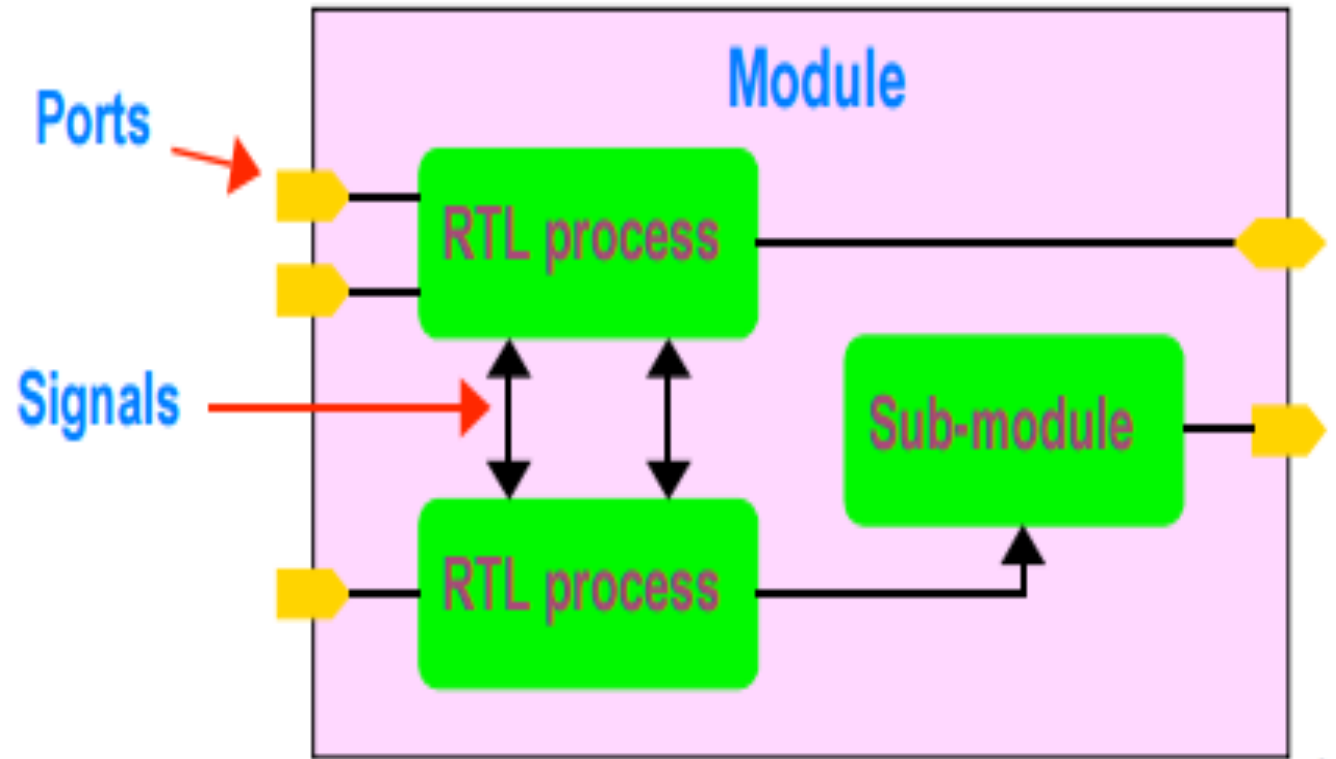
– `sc_in<sc_logic> a[32];` //input ports a[0] to a[31] of type sc_logic

Now, altogether...



- **Interface** defines (but does not implement) a set of access methods
- **Channel** implements the methods of one or more interfaces
- **Port** is bound to Channel through the Interface

Module



```
// SystemC
SC_MODULE(module_name){
    -----
}
```

```
// Verilog
module module_name();
    -----
endmodule
```

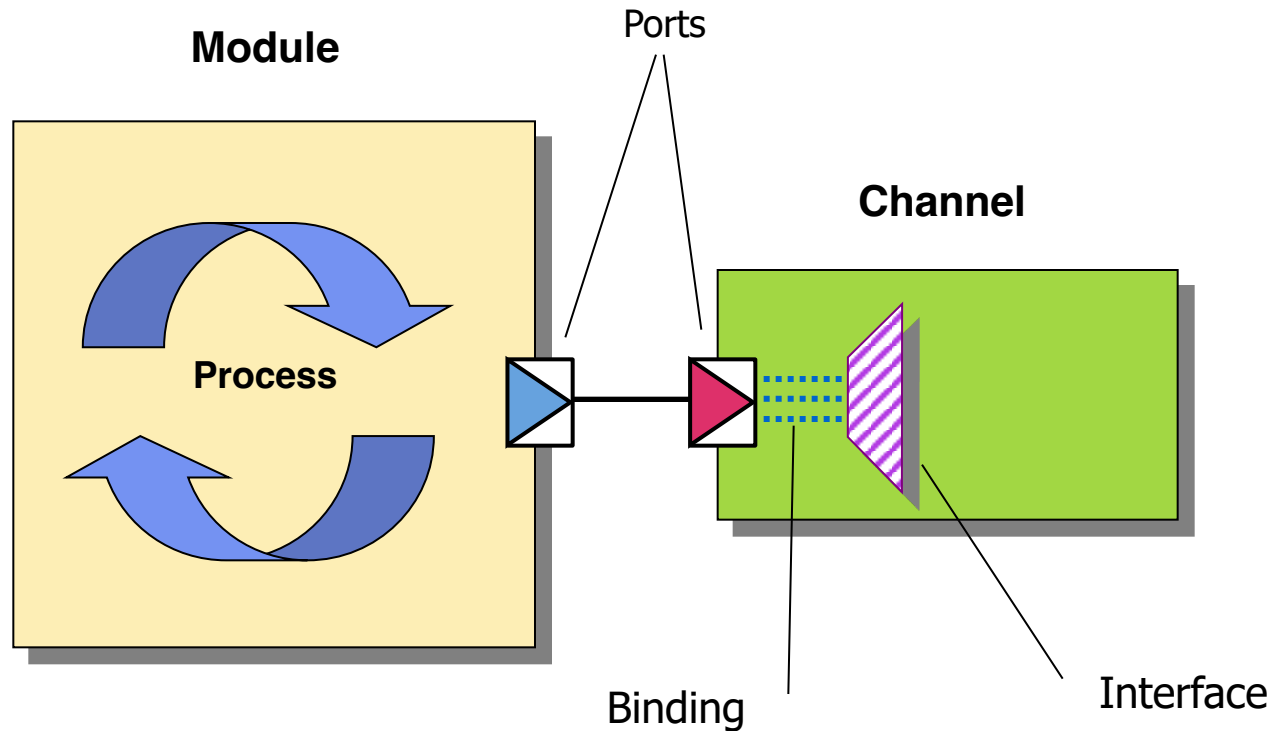
SystemC Module

- Basic building block to partition a complex design into smaller components
- **Module** is a C++ class.

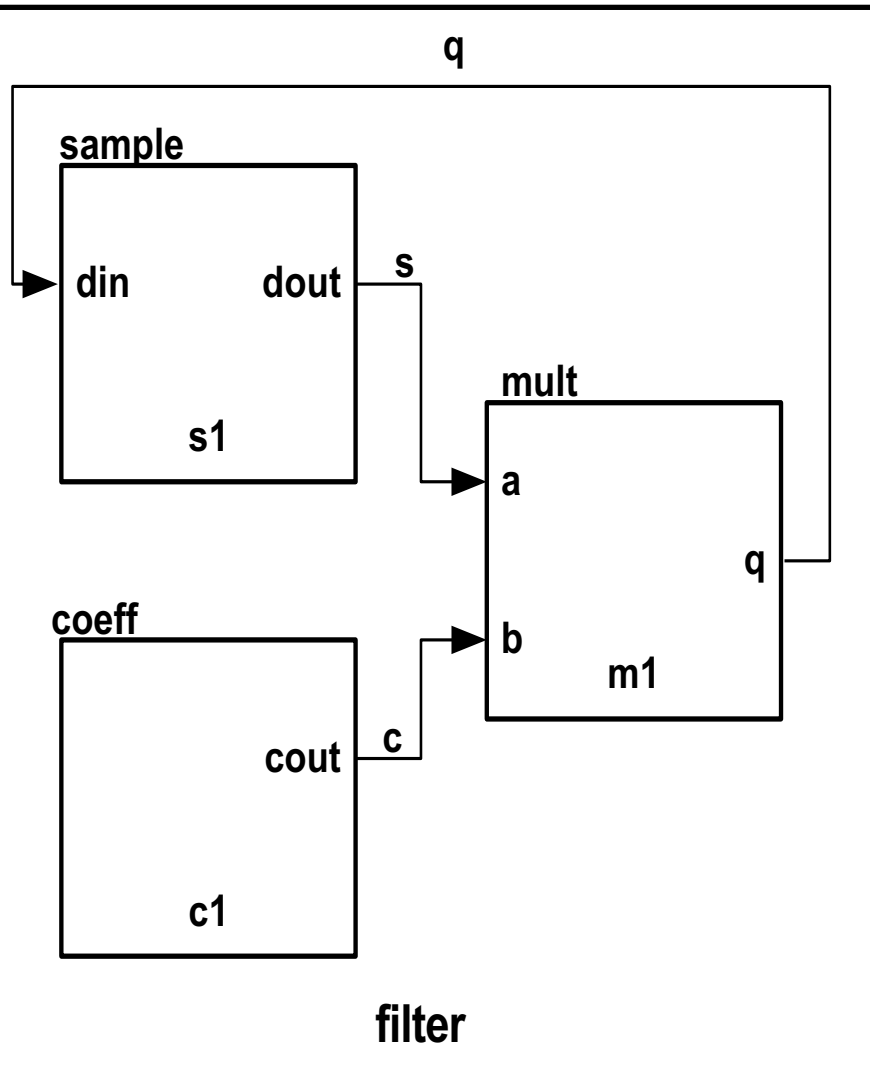


```
SC_MODULE (FIFO) {  
    //ports, process, internal data, etc  
    sc_in<bool> load;  
    sc_in<bool> read;  
    sc_inout<int> data;  
    sc_out<bool> full;  
    sc_out<bool> empty;  
  
    SC_CTOR(FIFO){  
        //body of constructor;  
        //process declaration, sensitivities, etc.  
    }  
};
```


Connecting **Module** to Channel



Modules



```
SC_MODULE(filter) {
```

```
    //Sub-modules : “components”
```

```
    sample *s1;
```

```
    coeff *c1;
```

```
    mult *m1;
```

```
    // Signals
```

```
    sc_signal<sc_uint 32> > q, s, c;
```

```
    //Constructor : “architecture”
```

```
    SC_CTOR(filter) {
```

```
    //Sub-modules instantiation and mapping
```

```
        s1 = new sample (“s1”);
```

```
        s1->din(q); //named mapping
```

```
        s1->dout(s);
```

```
        c1 = new coeff(“c1”);
```

```
        c1->out(c);
```

```
        m1 = new mult (“m1”);
```

```
        (*m1)(s, c, q); //Positional
```

```
        mapping
```

```
    }
```

```
}
```

Event and Process



- **Event**
- **Process**

Event

SystemC **Event**

➤ Basic synchronization object

➤ **Events**

- are used to synchronize processes
- are declared in a **module**

➤ Syntax

sc_event event_name1, event_name2, ... ;

Event Notification

- To trigger an event, call **notify()** method
- Notification of an event could be immediate or delayed

event_name.notify();

event_name.notify(SC_ZERO_TIME);

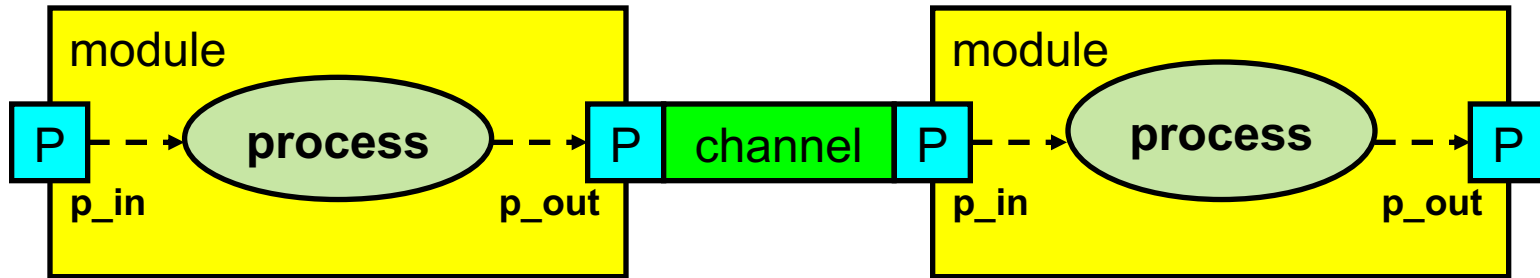
event_name.notify(1, SC_NS);

Process

```
// SystemC  
SC_METHOD(blk);  
    sensitive <<clk;  
  
void blk() {...}
```

```
// Verilog  
always @(clk)  
    begin : blk  
        ...  
    end
```

SystemC Process



- Module can have a number of Processes to describe its functionality
 - functions identified by the SystemC kernel
 - **invoked** by the kernel based on its **static** or **dynamic** sensitivity list
 - **cannot be called directly !!!**
- Two different types of processes:
 1. Threads: **SC_THREAD**
 2. Methods: **SC_METHOD**

Declaration of a Thread

- One (or more) **thread** process(es) may be declared C++ member function in a module

void process_name(void);

- Registered in the SystemC kernel within the module's constructor:

SC_THREAD(process_name);

- **Thread** process:
 - is implemented in an infinite loop
 - runs only when started by the SystemC scheduler
 - can be invoked based upon *Static* or *Dynamic* sensitivity list

Sensitivity List

➤ Static Sensitivity List

- Expressed in Module constructor
- Events such as **sc_signal** or **sc_event** that the process is sensitive to

void compute(void);

SC_THREAD(compute);

sensitive << irq1 << irq2; //in the constructor of a module

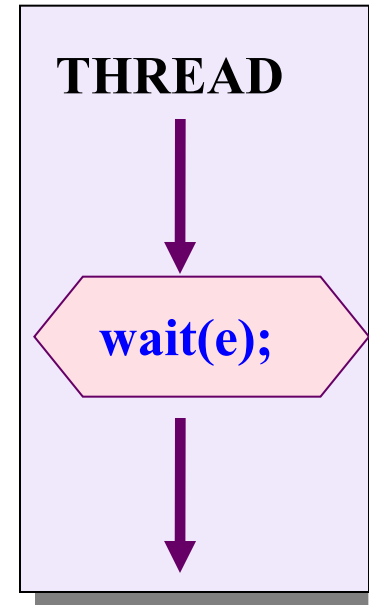
➤ Dynamic Sensitivity List

wait(irq);

//no explicit declaration in the constructor

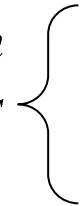
wait() Statement for a Thread

- Once (re)-activated, its statements are executed until a **wait()** statement is encountered
- Execution will be *suspended* upon hitting the **wait()** statement
- At its next reactivation, it will resume from the statement immediately following the **wait()**
- If no **wait()** is executed, the process will execute in zero time
- Examples:
 - **wait (e1| e2| e3)**
 - **wait (200, SC_NS)**
 - **wait (200, SC_NS, e1&e2&e3)**
 - **wait (SC_ZERO_TIME)**

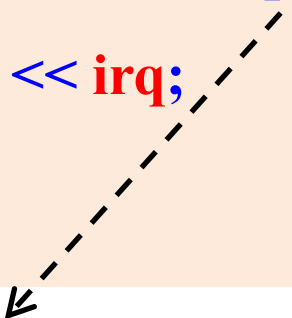


Example of a **Thread** Declaration

*Process declaration
within the module's
constructor!*



```
SC_HAS_PROCESS (traffic_generator);  
  
traffic_generator(sc_module_name, ...) {  
    void compute(void);  
    SC_THREAD(compute);  
    sensitive << irq;  
}
```



```
void compute(void) {  
    while(true) {  
        - - - -  
        wait(event);  
    }  
}
```

Method Process

What is it different from a [thread](#) process?

Similar to a [thread](#) process except that it [cannot be suspended](#)

Sensitivity List

- **Static Sensitivity:** Same as for thread processes
- **Dynamic Sensitivity:**
 - **`next_trigger(event);`**
 - **`next_trigger(time);`**

Declare, Invoke and Execute a **Method** Process

- Invoked by the SystemC scheduler whenever sensitive list changes
- Once invoked, the entire body of the process will be executed
- Upon completion, the **method** process returns the execution control back to the simulation kernel

*Process declaration
within the module's
constructor!*

```
SC_HAS_PROCESS (traffic_generator);  
traffic_generator(sc_module_name name, ...) {  
    void display (void);  
    SC_METHOD(display);  
    sensitive << irq;  
}  
  
void display (void {  
    - - - - -  
    next_trigger (event);  
}
```

A dashed arrow points from the **SC_METHOD(display);** line in the process declaration block to the **void display (void {** line in the function definition block, indicating that the method declaration refers to this function.

Method Processes

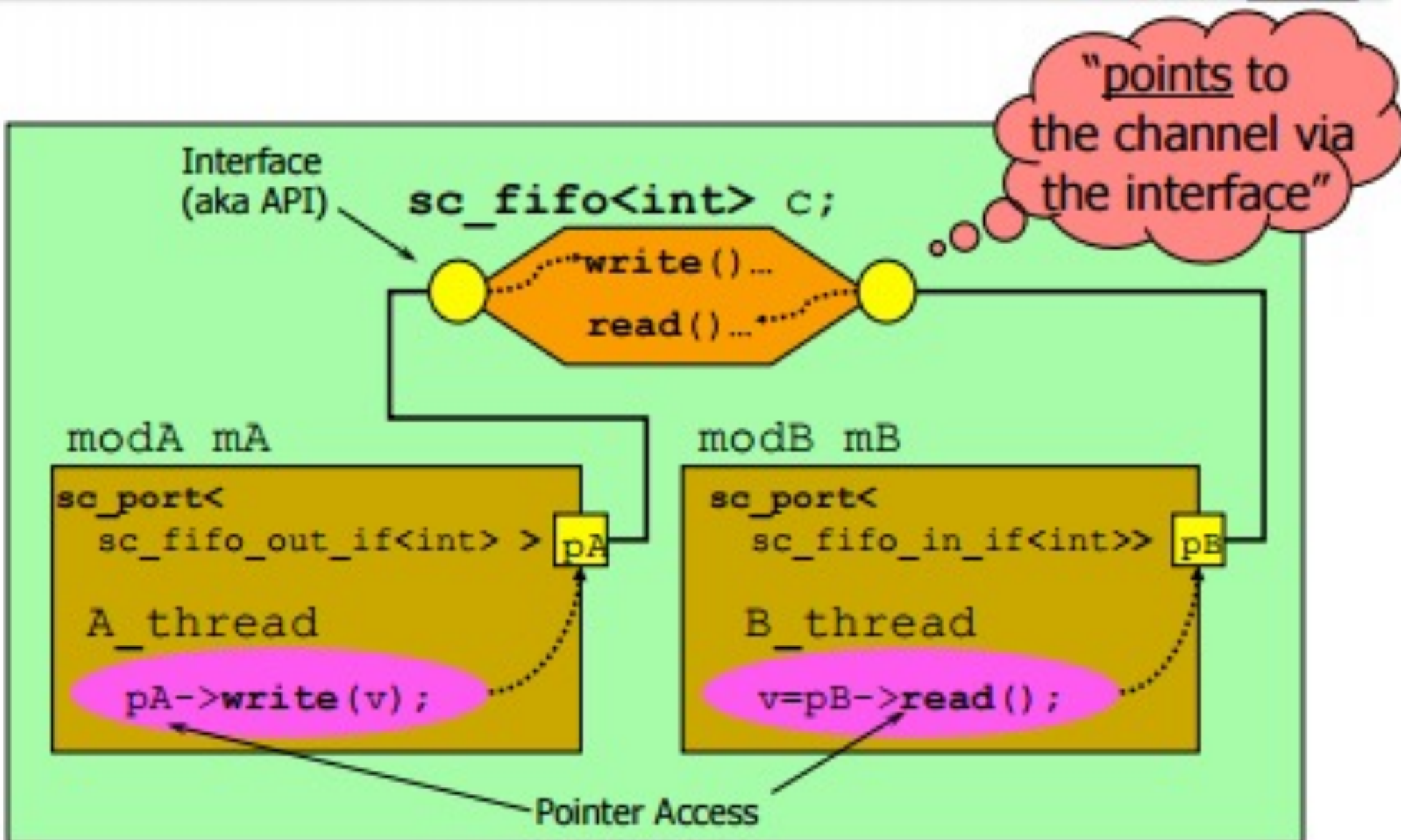
.h file

```
SC_MODULE(Mux21 ){  
    void doIt( void );  
    SC_CTOR( Mux21 ) {  
        SC_METHOD( doIt );  
        sensitive << selection;  
        sensitive << in1;  
        sensitive << in2;  
    }
```

.cpp file

```
void Mux21::doIt( void ) {  
    sc_uint<8> out_tmp;  
  
    if( selection.read() ) {  
        out_tmp = in2.read();  
    } else {  
        out_tmp = in1.read();  
    }  
    out.write( out_tmp );  
}
```

Now, altogether...



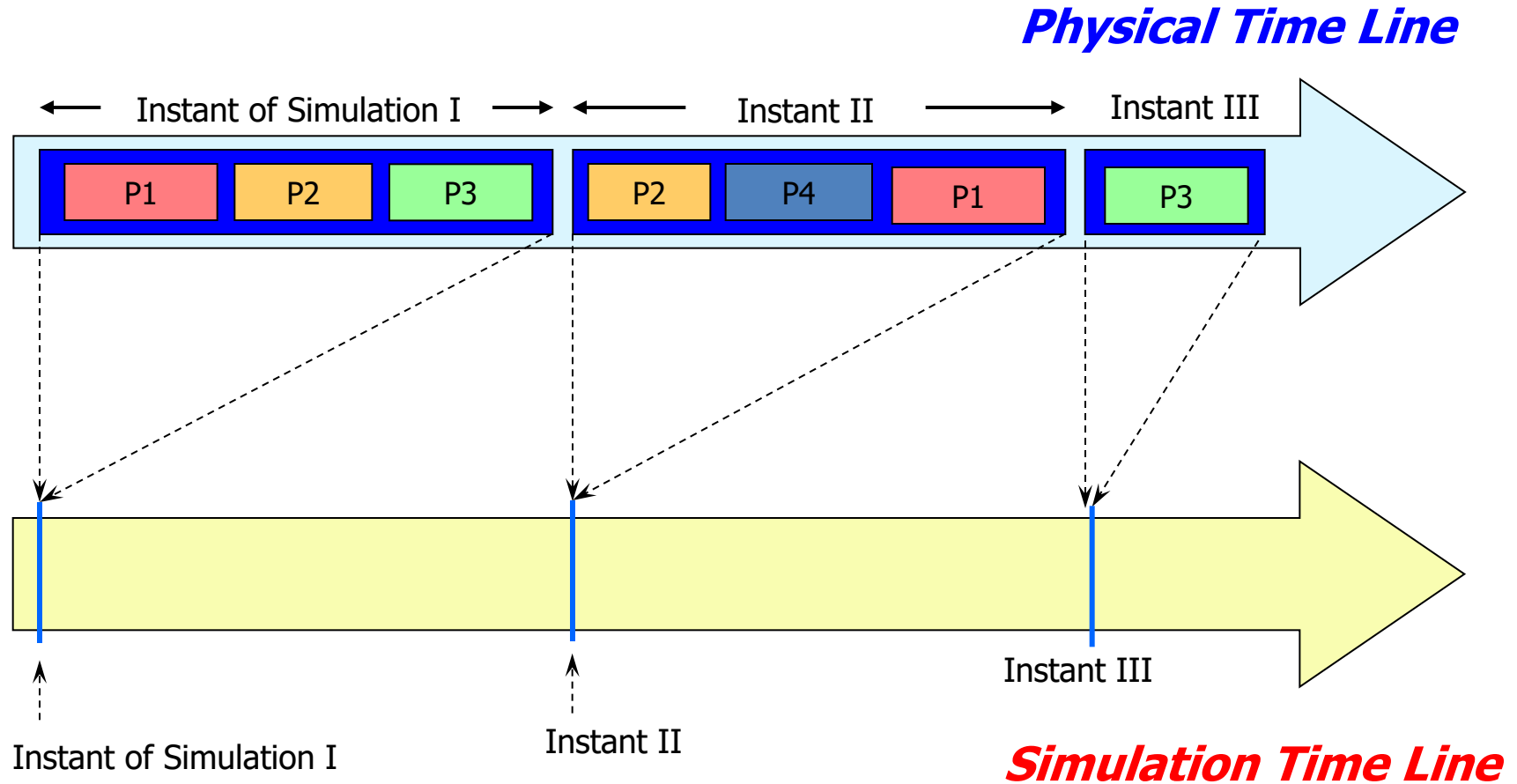
Simulator



- Time Concepts
- SystemC Synchronization
- SystemC Scheduler

Time Concepts

Physical Time vs. Simulation Time



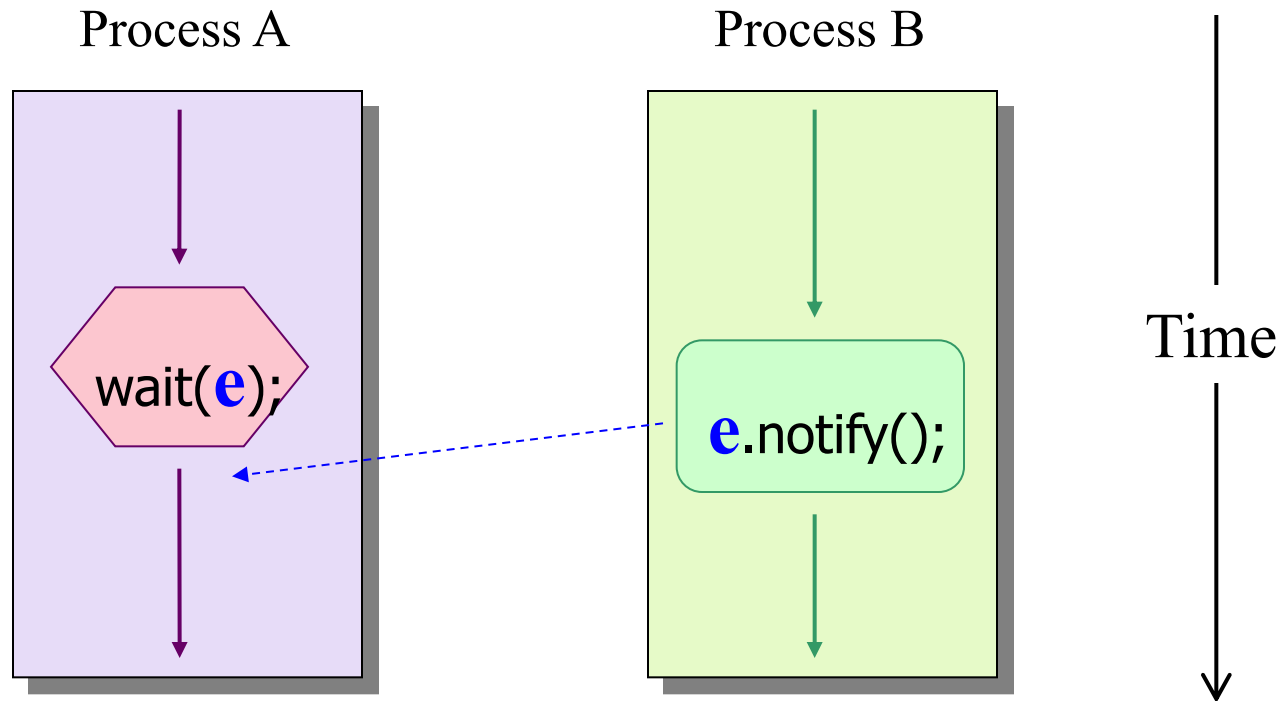
SystemC Synchronization

System Synchronization

Using SC_THREAD

- SystemC models concurrency
 - Synchronization primitives are thus needed
- To suspend a process in SystemC
 - **wait()**
- To resume the execution of a suspended process
 - **notify(...)**

Synchronizing by **Event** Notification

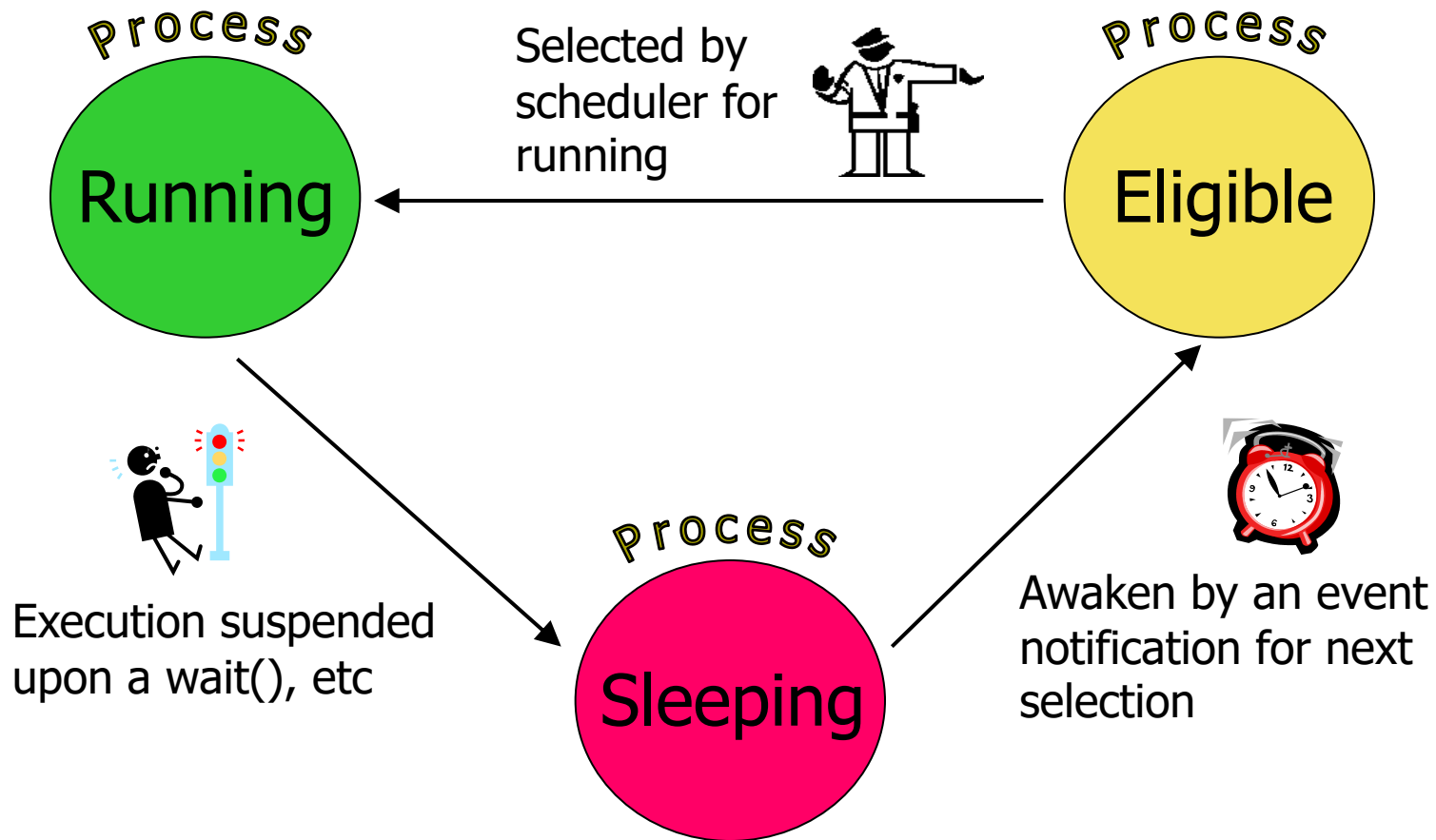


The process is suspended by the wait statement resumes its execution when the awaited event, *e*, is notified

SystemC Scheduler

Process Management

by a *non-preemptive scheduler*



SystemC: Execution Flowchart

// Entry point for program execution

void **sc_main** () { ← - - - **Program Entry** **Elaboration Starts**

 sc_signal timer1_sig, timer2_sig;
 my_channel channel("Channel1");

// Declaration and instantiation of the platform components

 Timer timer1("T1");

 Timer timer2("T2");

 Itc itc("ITC1");

// Bind the modules to the communication channel using ports

 timer1.sigoutport(timer1_sig);

 timer1.chanport(channel);

 timer2.chanport(channel);

 timer2.sigoutport(timer2_sig);

 itc.in1(timer1_sig);

 itc.in2(timer2_sig);

 itc.chanport(channel);

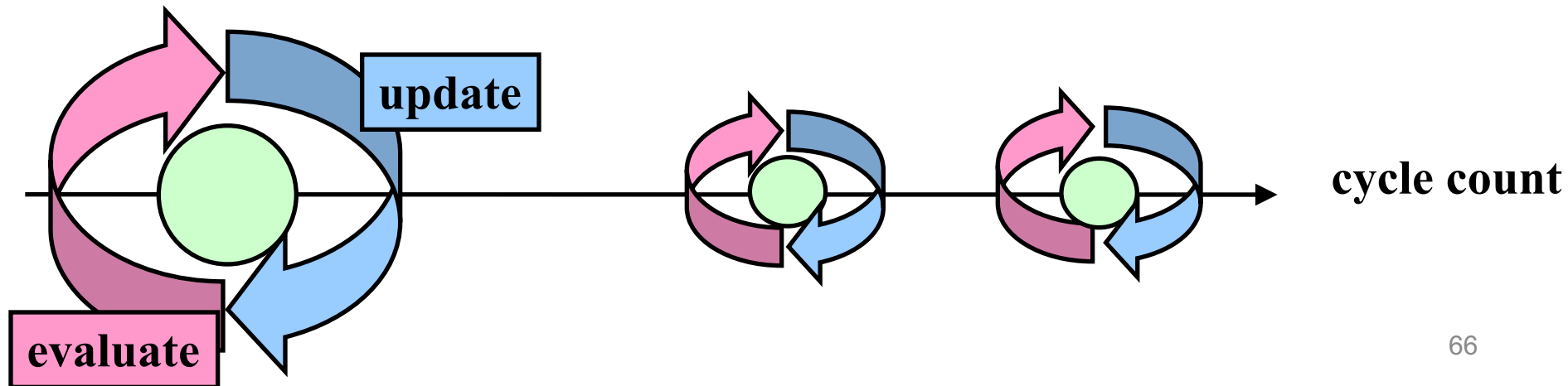
sc_start(-1); // -1 = runs indefinitely

Simulation Starts - - - →

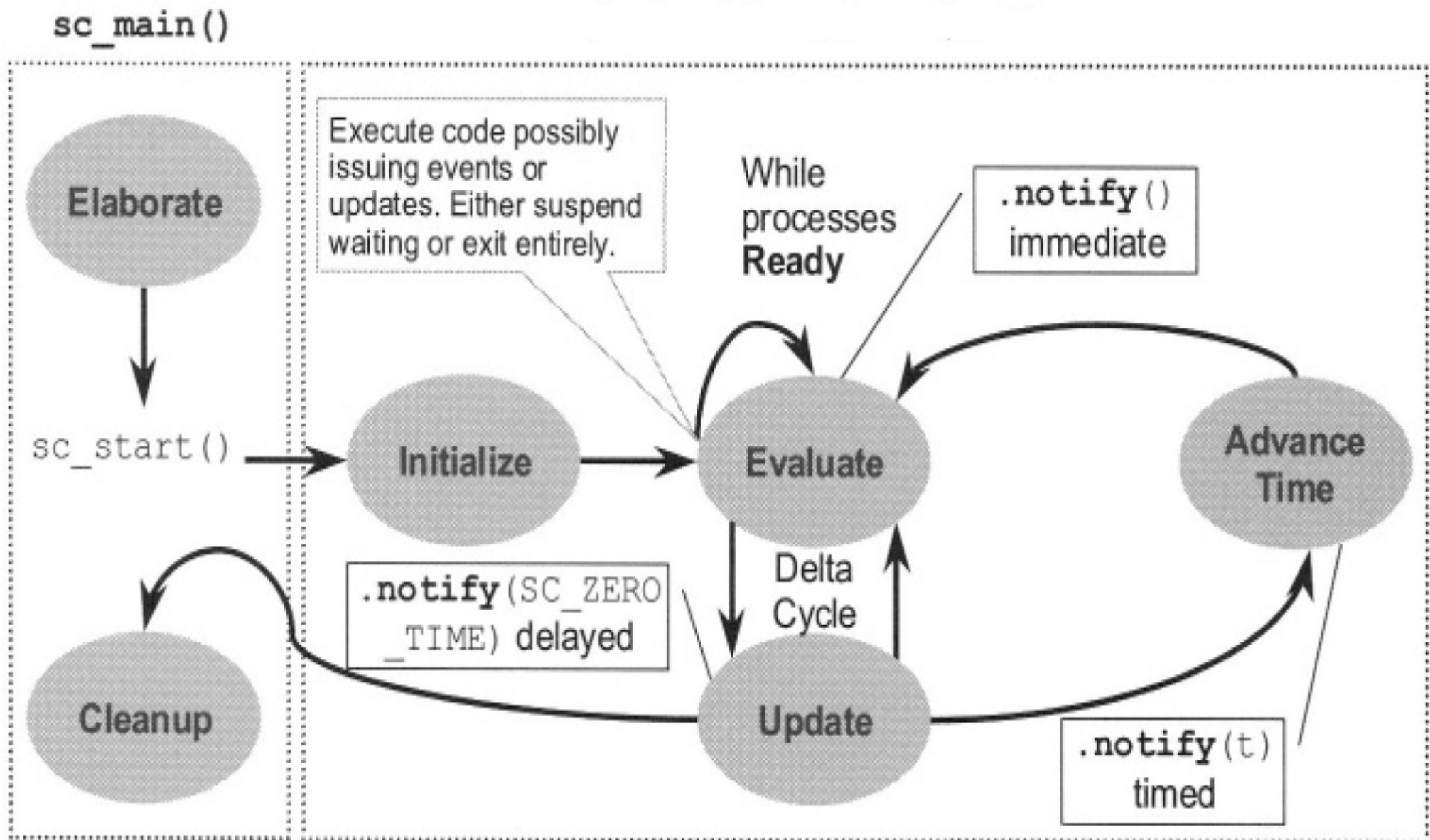
Elaboration Ends

The Simulation **Delta Cycle**

Scenario	Multiple independent but intercommunicating processes
Problem	The processes are executed concurrently but in a non-deterministic order that can give rise to race conditions.
Solution	At each simulation “cycle” withhold process outputs until all processes again attain a “resting” state.
Implementation	Delta cycles have separate evaluate and update phases: ◆ Evaluate Execute all “runnable” process to resting state deferring the changes/ updates made by them ◆ Update Make available those deferred changes/ updates



Process Management – Simulation Flowchart



```

SC_MODULE(my_mod) {
    void P1(void);
    void P2(void);
    sc_port<sc_signal_inout_if<sc_logic>> port;
    sc_event event;
    SC_CTOR(my_mod) {
        SC_THREAD(P1);
        SC_THREAD(P2);
        port.initialize('Z');
    }
}.

```

```

void my_mod::P1(void) {
    port->write('1');
    wait(SC_ZERO_TIME);
    cout<<port->read();
    port->write('2');
    event.notify();
    cout<<port->read();
    wait(10,SC_NS);
    cout<<port->read();
}

```

```

void my_mod::P2(void) {
    cout<<port->read();
    wait(event);
    cout<<port->read();
}

```

0ns+delta1	Evaluate	run P1
0ns+delta1	Evaluate	run P2
0ns+delta1	Update	update port with '1'
0ns+delta2	Evaluate	Run P1=> event notification will make P2 runnable in current evaluate phase.
0ns+delta2	Evaluate	Run P2 P2 comes to end &exit.
0ns+delta2	Update	Update port with '2'
0ns+delta2	Update	simulation time advances to 10ns
10ns+delta1	Evaluate	Run P1 P1 comes to end & exit
Simulation stopped: No runnable thread/ Event		

Example of SystemC Models



- Example of SystemC IP
- Example of SystemC System

Example of SystemC IP

D Flip-Flop

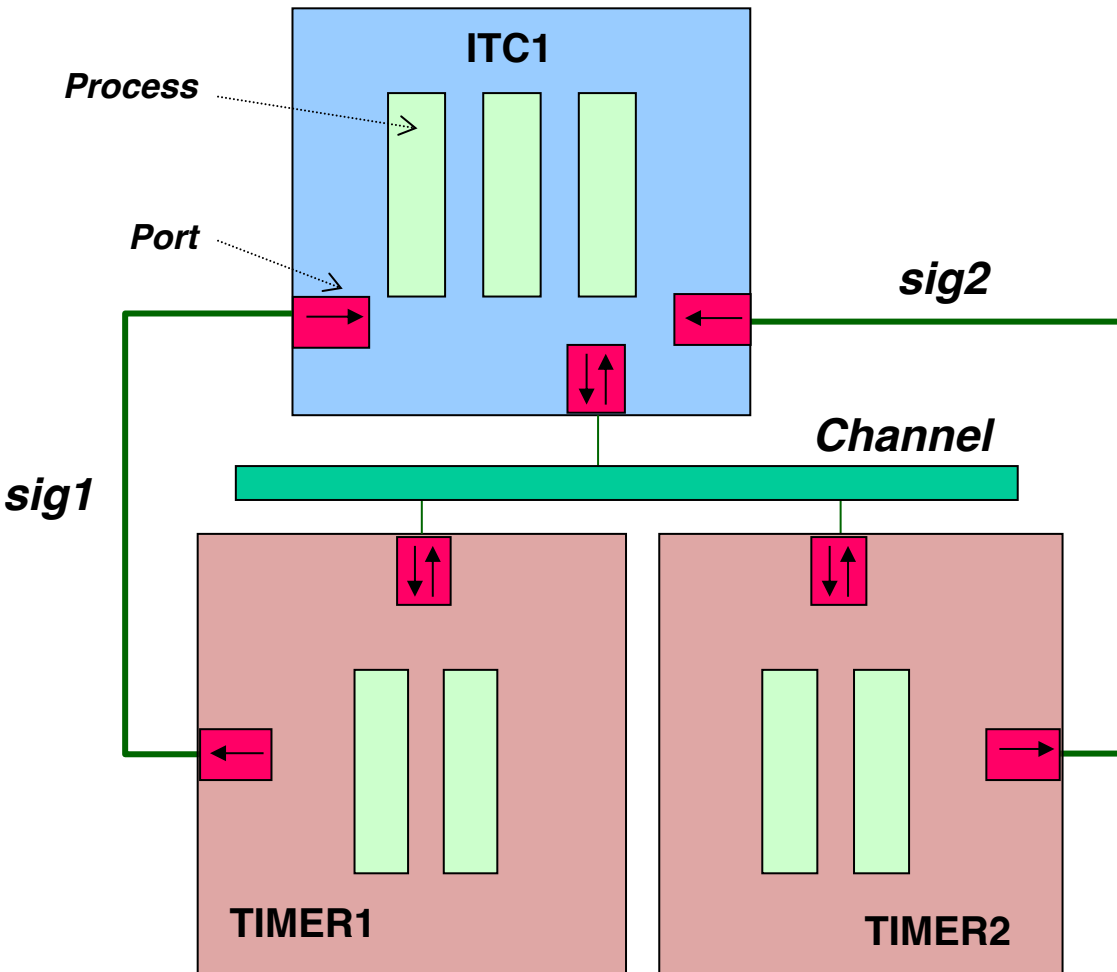
```
#include "systemc.h"

SC_MODULE (dff) {
    sc_in<bool> din; // Declaration of ports
    sc_in<bool> clock;
    sc_out<bool> dout;
    void doit(void); // Declaration of the process
    // Constructor
    SC_CTOR (dff) {
        // registering process with kernel
        SC_METHOD (doit); ----->
        sensitive_pos << clock ;// Sensitivity list
    }
};
```

```
//dff.cpp
#include "dff.h"
// process implementation
void dff::doit();
dout = din;
}
```

Example of SystemC System

A Typical System



ITC1

- an object, i.e. instance of the class ITC

TIMER1 and TIMER2

- 2 objects, i.e. instances of the class TIMER

sig1 and sig2

- 2 objects, i.e. instances of the class `sc_signal`

Steps to develop SystemC System

- Write **sc_main**
- **Instantiate** the components of the system
- **Bind** the modules to the communication channels through port
- Start the simulation by calling **sc_start(args)**

Example Code

// Entry point for the program execution

```
void sc_main () {
```

```
    sc_signal<bool> timer1_sig, timer2_sig;  
    my_channel channel("Channel1");
```

// Declaration and instantiation of the platform components

```
    Timer timer1("T1");
```

```
    Timer timer2("T2");
```

```
    Itc itc("ITC1");
```

//Bind the modules to the communication channel using ports

```
    timer1.sigoutport(timer1_sig);
```

```
    timer1.chanport(channel);
```

```
    timer2.chanport(channel);
```

```
    timer2.sigoutport(timer2_sig);
```

```
    itc.in1(timer1_sig);
```

```
    itc.in2(timer2_sig);
```

```
    itc.chanport(channel);
```

```
    // Starting the simulation
```

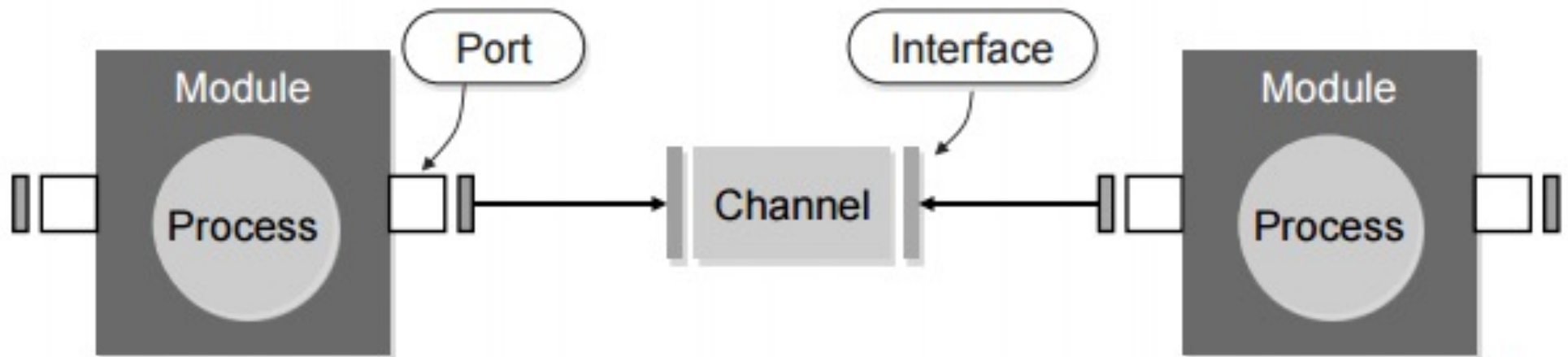
← - - - Program Entry

↑
instantiation
↓

↑
Binding
↓

SystemC is

- Modules (structure)
- Ports (structure)
- Processes (computation, concurrency)
- Channels (communication)
- Interfaces (communication refinement)
- Events (time, scheduling, synchronisation)
- Data types (hardware, fixed point)



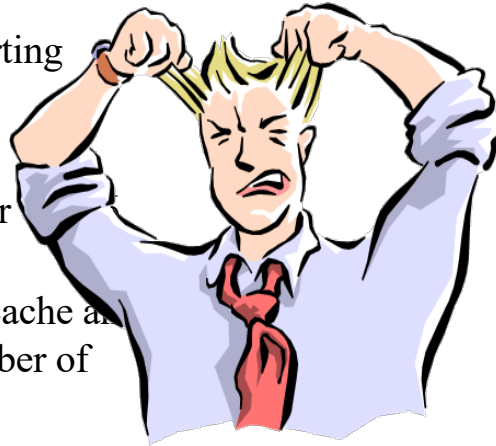
SystemC Resources

- Standard SystemC Language Reference Manual
 - IEEE Std. 1666-2011 <http://www.ieee.org>
- Accellera Systems Initiative <http://www.accellera.org>
 - Technical Papers (Application Notes, Documentation, Tips), News, Events, Products & Solutions, Discussion, FAQs
- SystemC Publications
 - Jayram Bhasker. "A SystemC Primer". [Star Galaxy Publishing](#), 2004.
 - David Black, (et al.). "SystemC: From The Ground Up". [Springer](#), 2010.
 - **Thorsten Grötzer, (et al.). "System Design with SystemC". [Springer](#), 2002.**
 - Frank Ghenassia, (Ed.). "Transaction-Level Modeling with SystemC". [Springer](#), 2005.
 - W. Müller; W. Rosenstiel; J. Ruf (Eds.). "SystemC Methodologies and Applications". [Springer](#), 2003.

TLM

Complexity of Designs

- Let's take an example:
 - Design a quad core SoC where each core
 - ...
 - ... has a multistage-stage superscalar out-of-order pipeline.
 - ... has 8 threads of execution supporting speculative loads and stores
 - ... supports precise exception
 - ... has a tournament branch predictor
 - ... has a separate L1 D-cache, L1 I-cache and a unified L2 cache whose size, number of blocks per line, and associativity is parameterizable.



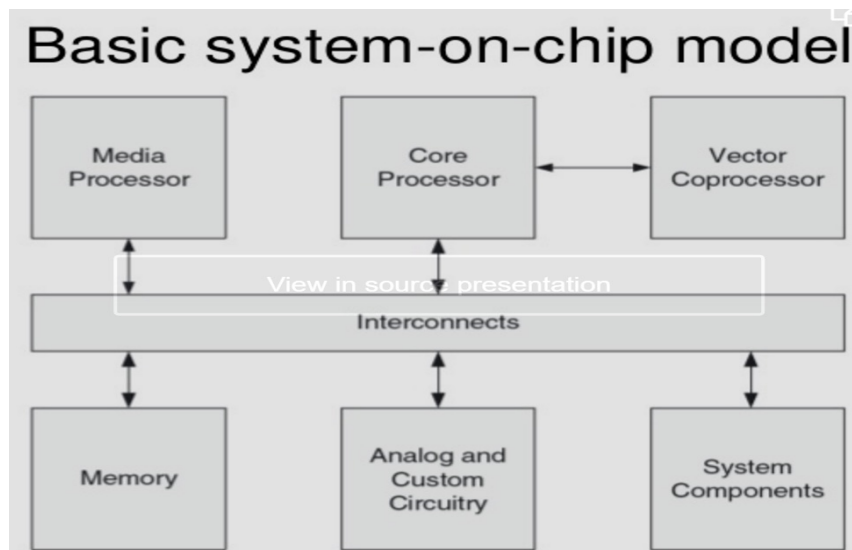
Complexity of Verification

- What if there is a Load operation which has a TLB miss, a cache miss and a page fault, and is speculatively executing on a conditional branch which is mispredicted?
- Generate a test case?

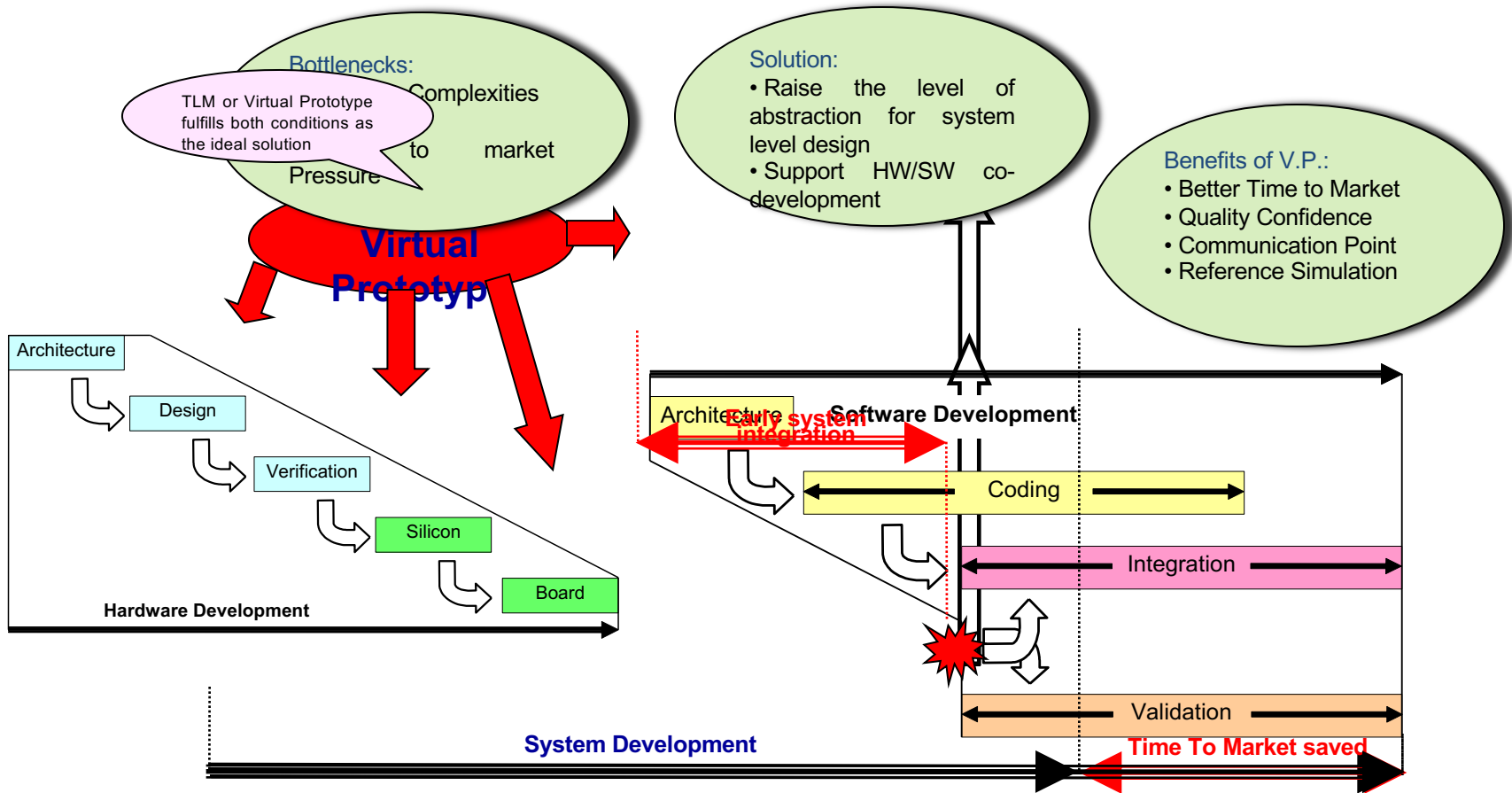


Introduction : System On Chip

- *How does a system-on-chip like today?*



System On Chip Design Flow

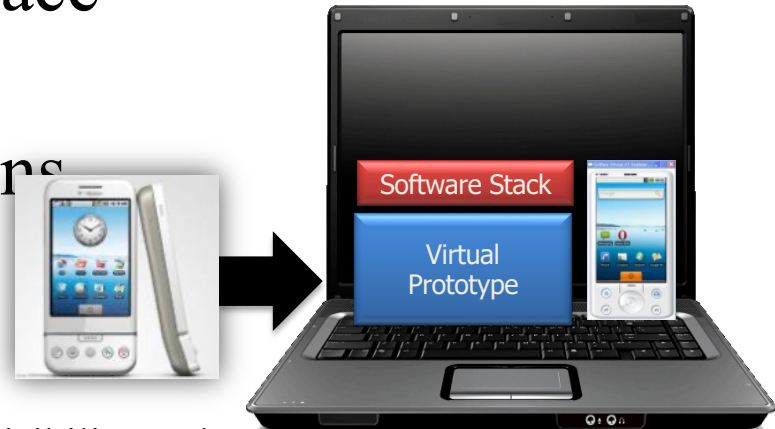


High Level Modeling



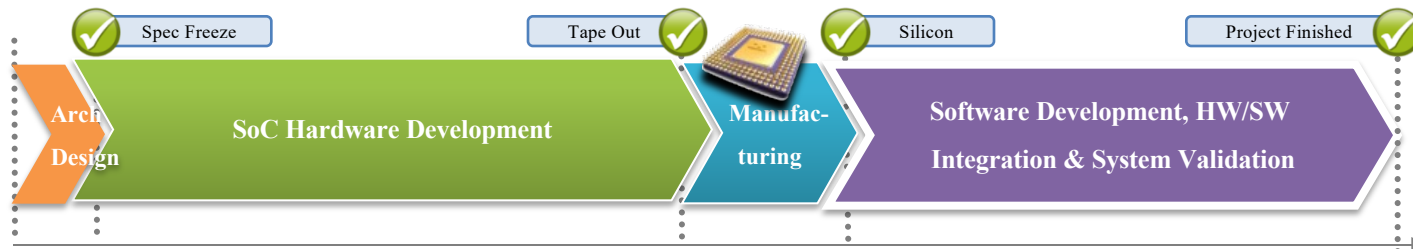
What is a Virtual Prototype?

- Fully functional software model of complete systems
 - SoC, board, I/O, user interface
- Executes unmodified production code
 - Drivers, OS, and applications
- Runs fast
 - Boots OS in seconds
- Highest debugging efficiency through full system visibility and control
 - Supports multi-core SoCs debug
- Easy deployment

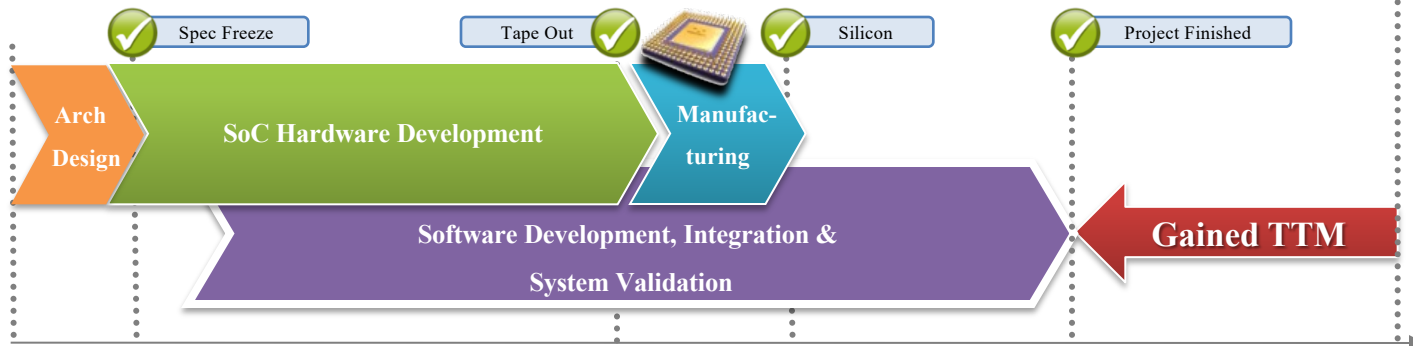


Why Prototype?

Without Prototyping



With Prototyping

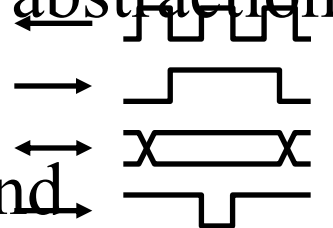
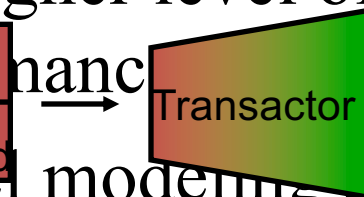
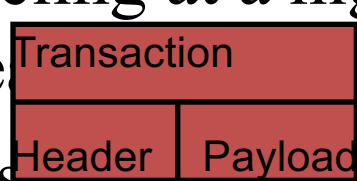


Transaction Level Modeling and Verification

- Yet another RTL modeling language is not interesting.
- What makes SystemC interesting?

– Modeling at a higher level of abstraction

incre



– Transaction level modeling and

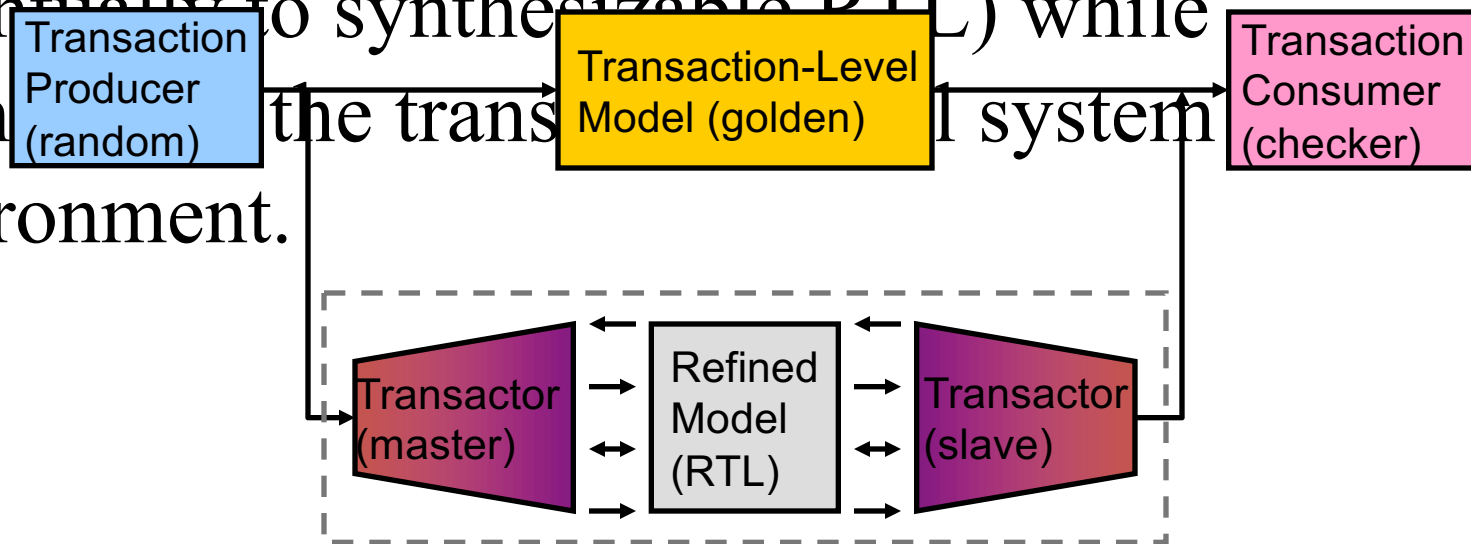
verification increases your productivity

A transactor adapts between a transaction level interface and a signal level interface

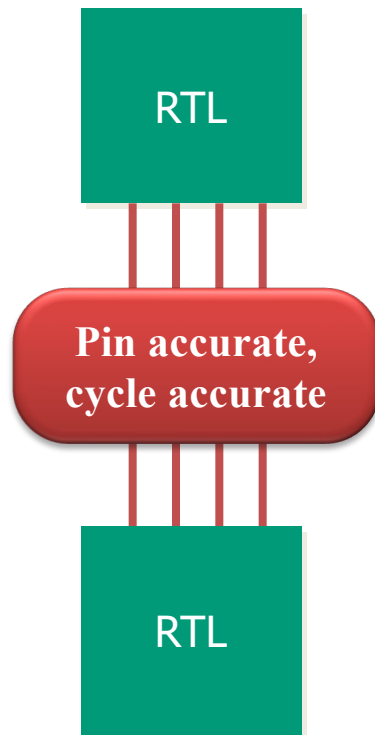
- Less detail means higher performance and easier debug

Refining the Transaction Level Model

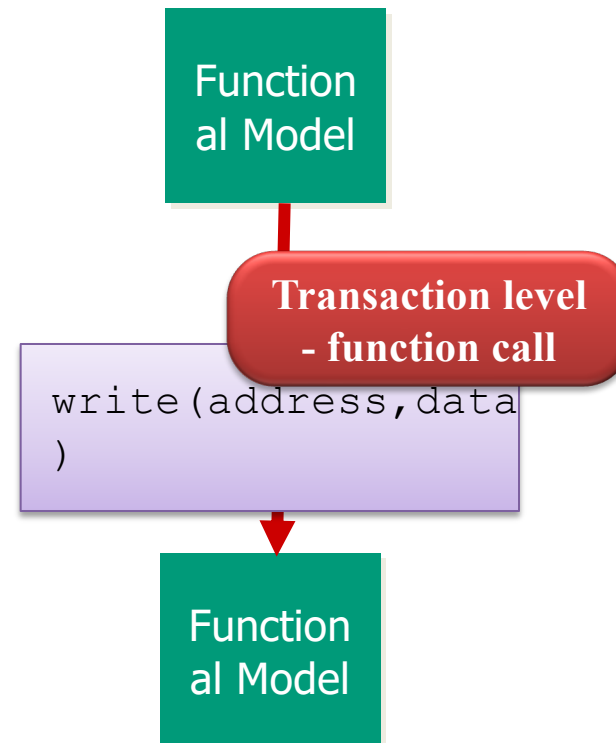
- You refine the transaction level model (eventually to synthesizable RTL) while maintaining the transaction level system environment.



Transaction Level Modeling 101

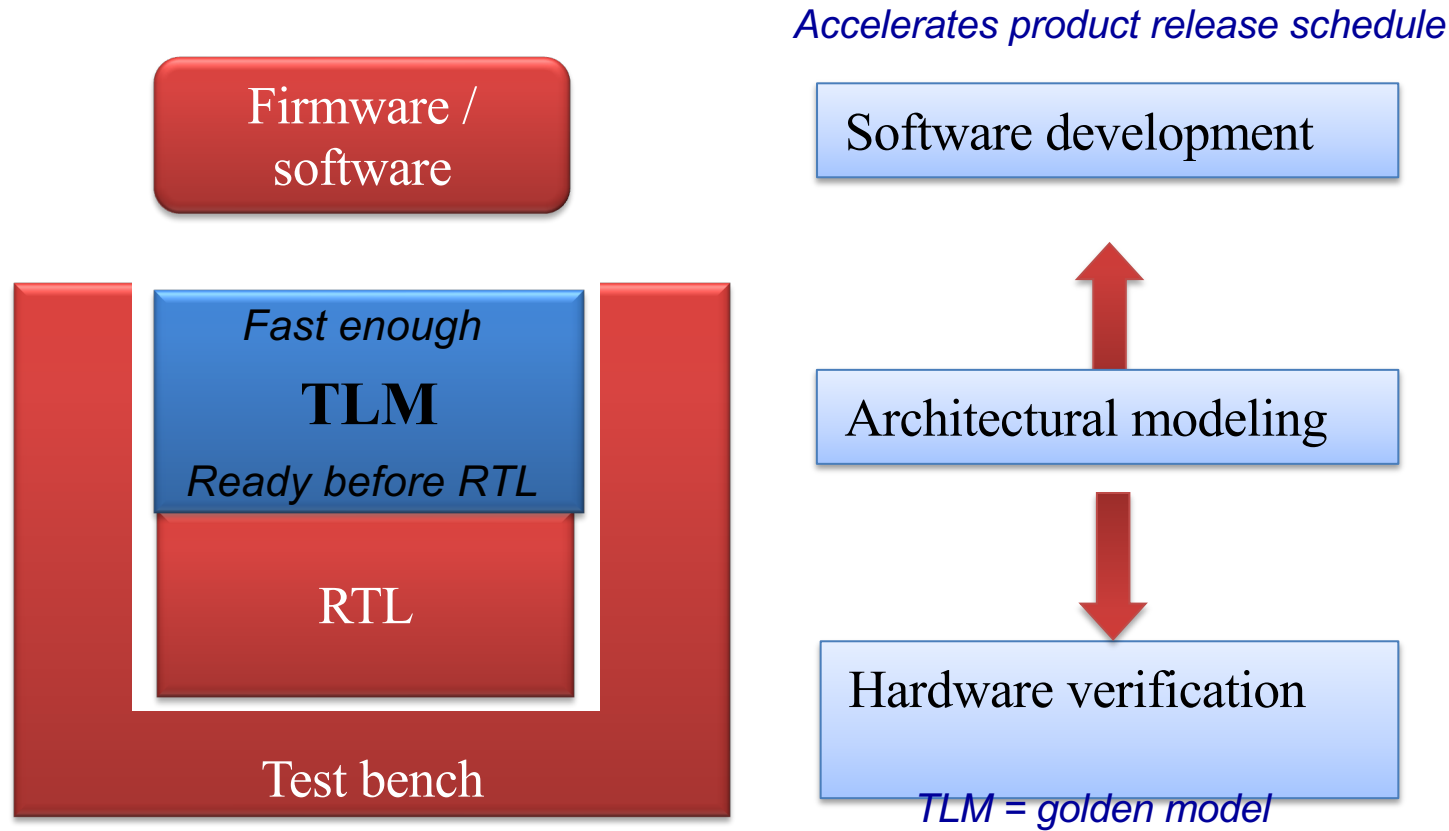


Simulate every event



100-10,000 X faster simulation

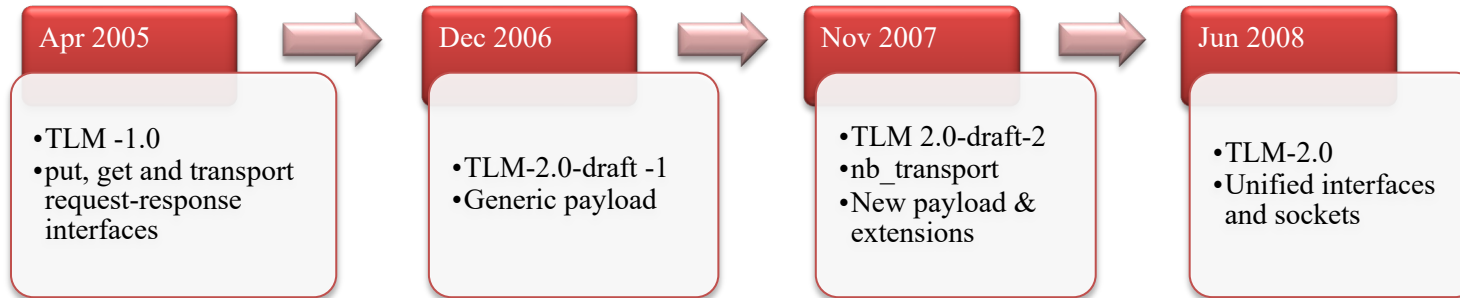
Reasons for Using TLM



Typical Use Cases for TLM

- Represents key architectural components of hardware platform
- Architectural exploration, performance modelling
- Software execution on virtual model of hardware platform
- Golden model for hardware functional verification
- Available before RTL
- Simulates much faster than RTL

History of TLM



TLM-2 Requirements

- Transaction-level memory-mapped bus modelling
- Register accurate, functionally complete
- Fast enough to boot software O/S in seconds
- Loosely-timed and approximately-timed modelling
- Interoperable API for memory-mapped bus modelling
- Generic payload and extension mechanism
- Avoid adapters where possible

Use Cases, Coding Styles and Mechanisms

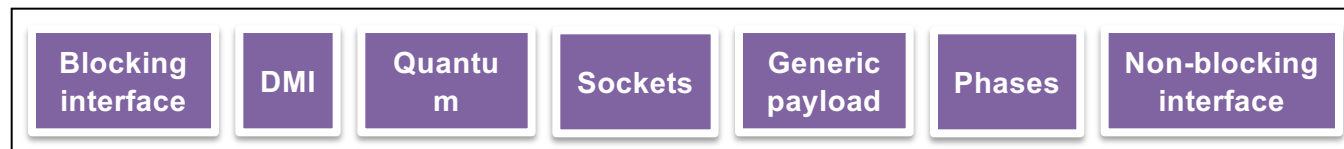
Use cases



TLM-2 Coding styles



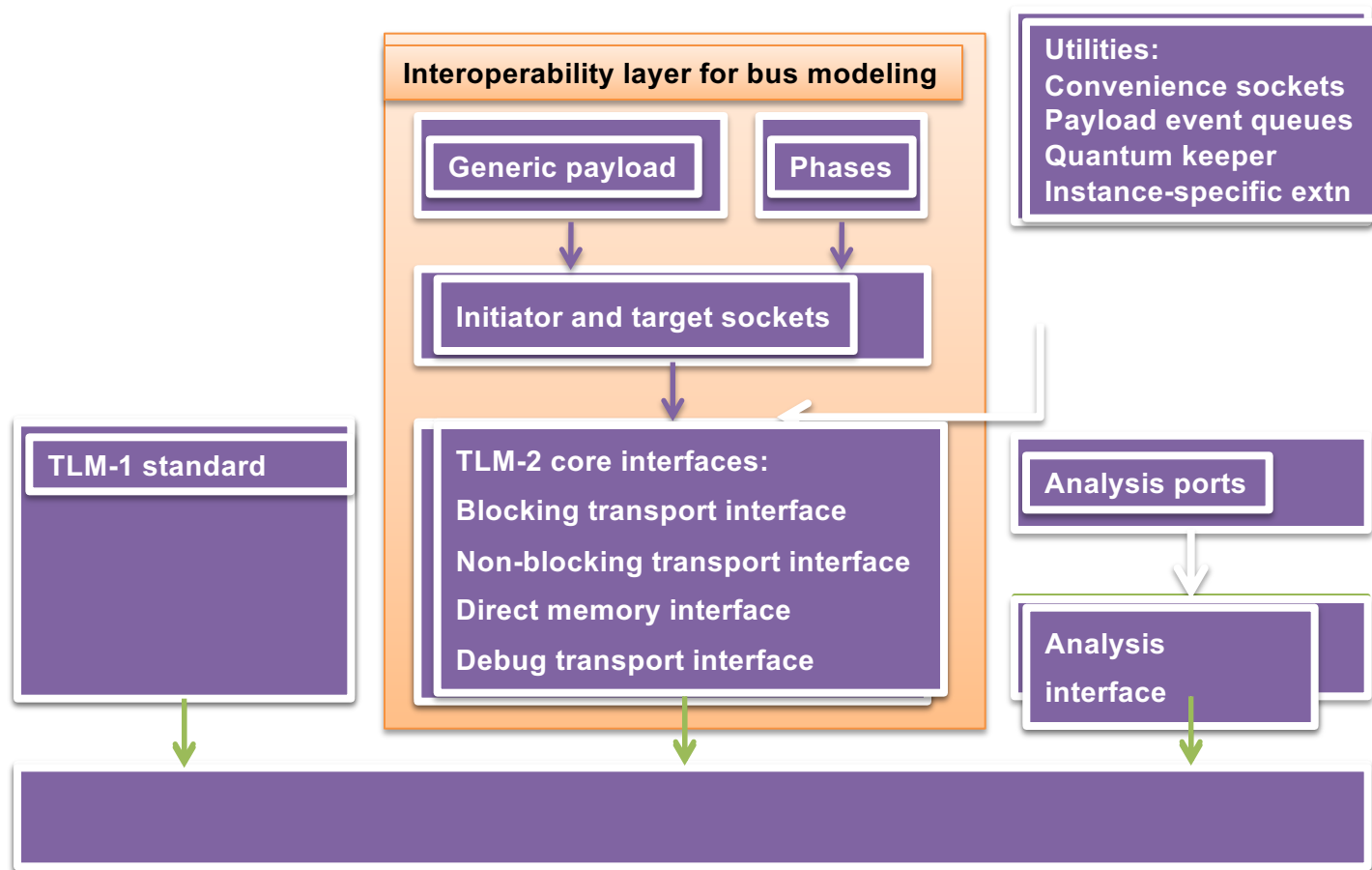
Mechanisms



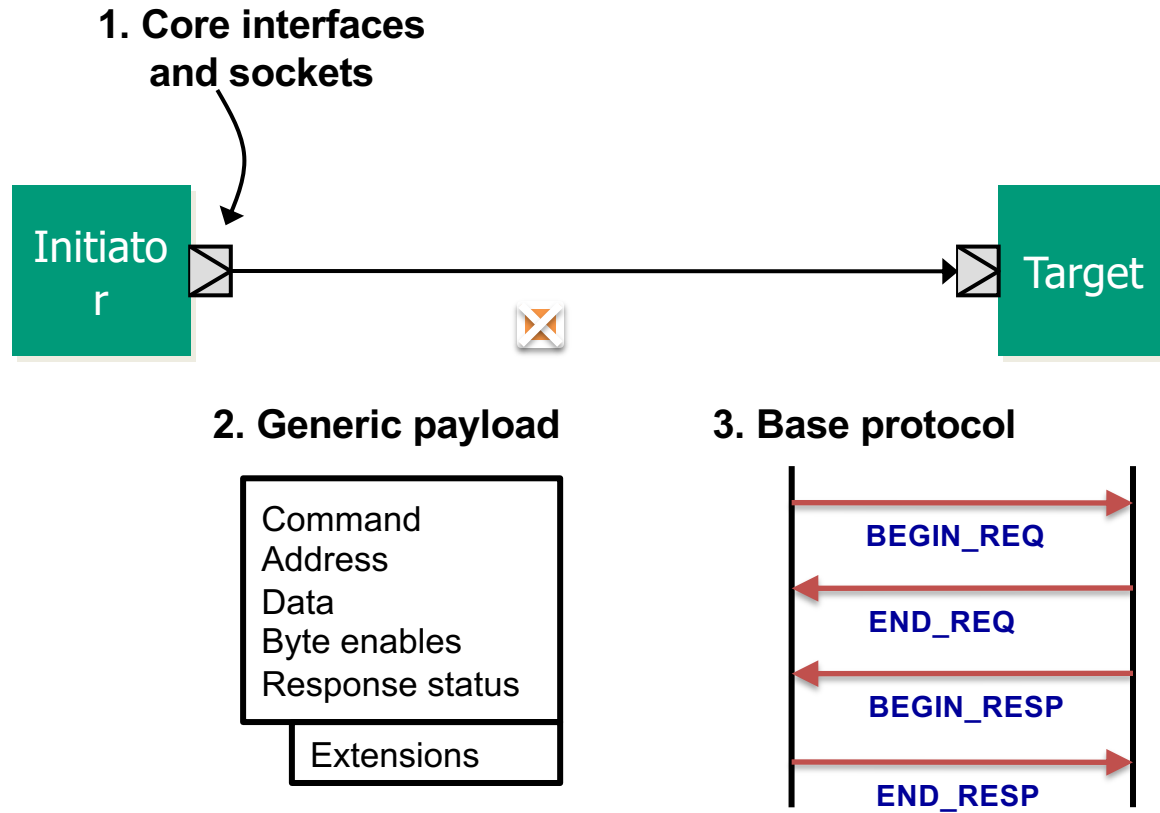
Coding Styles

- Loosely-timed = as fast as possible
 - Only sufficient timing detail to boot O/S and run multi-core s
 - Processes can run ahead of simulation time (temporal decoupling)
 - Each transaction has 2 timing points: *begin* and *end*
 - Uses direct memory interface (DMI)
- Approximately-timed = just accurate enough for performance modeling
 - *aka* cycle-approximate or cycle-count-accurate
 - Sufficient for architectural exploration

The TLM 2.0 Classes



Interoperability Layer



Maximal interoperability for memory-mapped bus models

Directory Structure

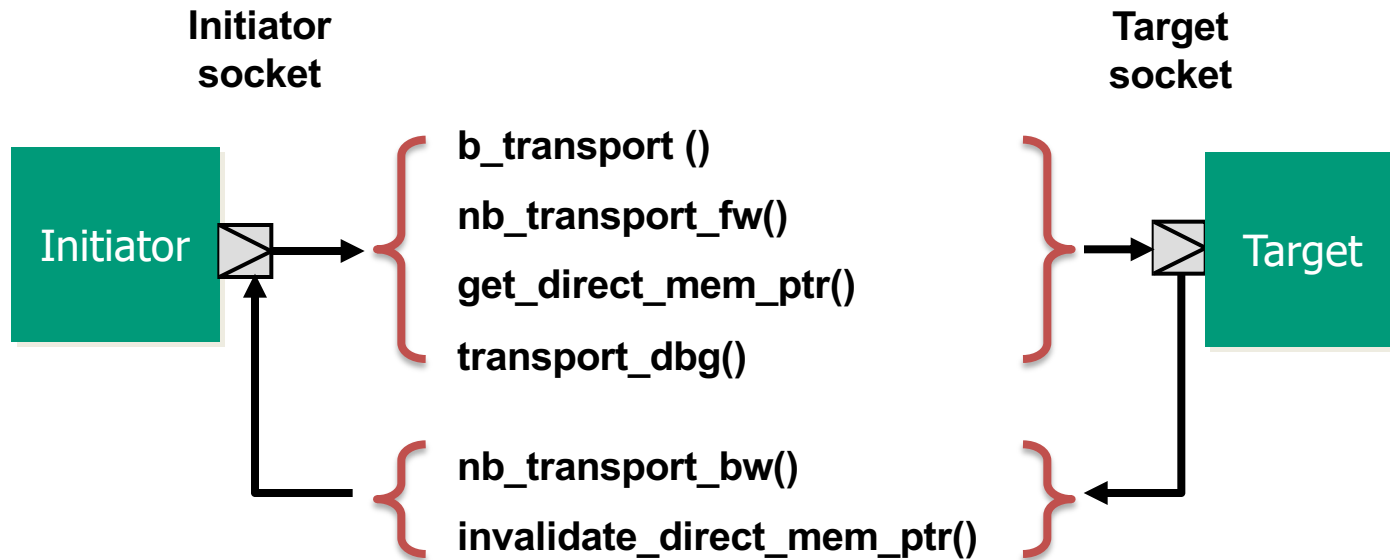
```
include/tlm
    tlm_h
        tlm_req_rsp          TLM-1.0 legacy
        tlm_trans            TLM-2 interoperability classes
            tlm_2_interfaces TLM-2 core interfaces
            tlm_generic_payload TLM-2 generic payload
            tlm_sockets      TLM-2 initiator and target sockets
        tlm_quantum          TLM-2 global quantum
        tlm_analysis         TLM-2 analysis interface, port, fifo
    tlm_utils               TLM-2 utilities

docs
    doxygen
examples
unit_test
```

Utilities

- tlm_utils
 - Convenience sockets
 - Payload event queues
 - Quantum keeper
 - Instance-specific extensions
- Productivity
- Shortened learning curve
- Consistent coding style
- *Not* part of the interoperability layer

Initiator and Target Sockets



[Link](#)

Sockets provide fw and bw paths and group interfaces

Benefit of Sockets

- Group the transport, DMI and debug transport interfaces
 - Bind forward and backward paths with a single call
 - Strong connection checking
 - Have a bus width parameter
-
- Using core interfaces without sockets is not recommended

TLM-2 Core Interfaces - Transport

tlm_blocking_transport_if

```
void b_transport( TRANS& , sc_time& );
```

tlm_fw_nonblocking_transport_if

```
tlm_sync_enum nb_transport_fw( TRANS& , PHASE& ,  
sc_time& );
```

tlm_bw_nonblocking_transport_if

```
tlm_sync_enum nb_transport_bw( TRANS& , PHASE& ,  
sc_time& );
```

TLM-2 Core Interfaces - DMI and Debug

tlm_fw_direct_mem_if

```
bool get_direct_mem_ptr( TRANS& trans , tlm_dmi&  
dmi_data ) ;
```

tlm_bw_direct_mem_if

```
void invalidate_direct_mem_ptr( sc_dt::uint64 start_range,  
                                sc_dt::uint64 end_range )  
;
```

tlm_transport_dbg_if

```
unsigned int transport_dbg( TRANS& trans ) ;
```

May all use the generic payload transaction type

Blocking versus Non-blocking

Transport

LT

AT

Socket
s

- Blocking transport interface
 - Includes timing annotation
 - Typically used with loosely-timed coding style
 - Forward path only
- Non-blocking transport interface
 - Includes timing annotation and transaction phases
 - Typically used with approximately-timed coding style
 - Called on forward and backward paths
- Share the same transaction type for interoperability

Blocking Transport

LT

Blocking
Transport

Transaction type



```
template < typename TRANS = tlm_generic_payload >
```

```
class tlm_blocking_transport_if : public virtual sc_core::sc_interface {  
public:
```

```
    virtual void b_transport ( TRANS& trans , sc_core::sc_time& t )  
        = 0;
```

```
};
```



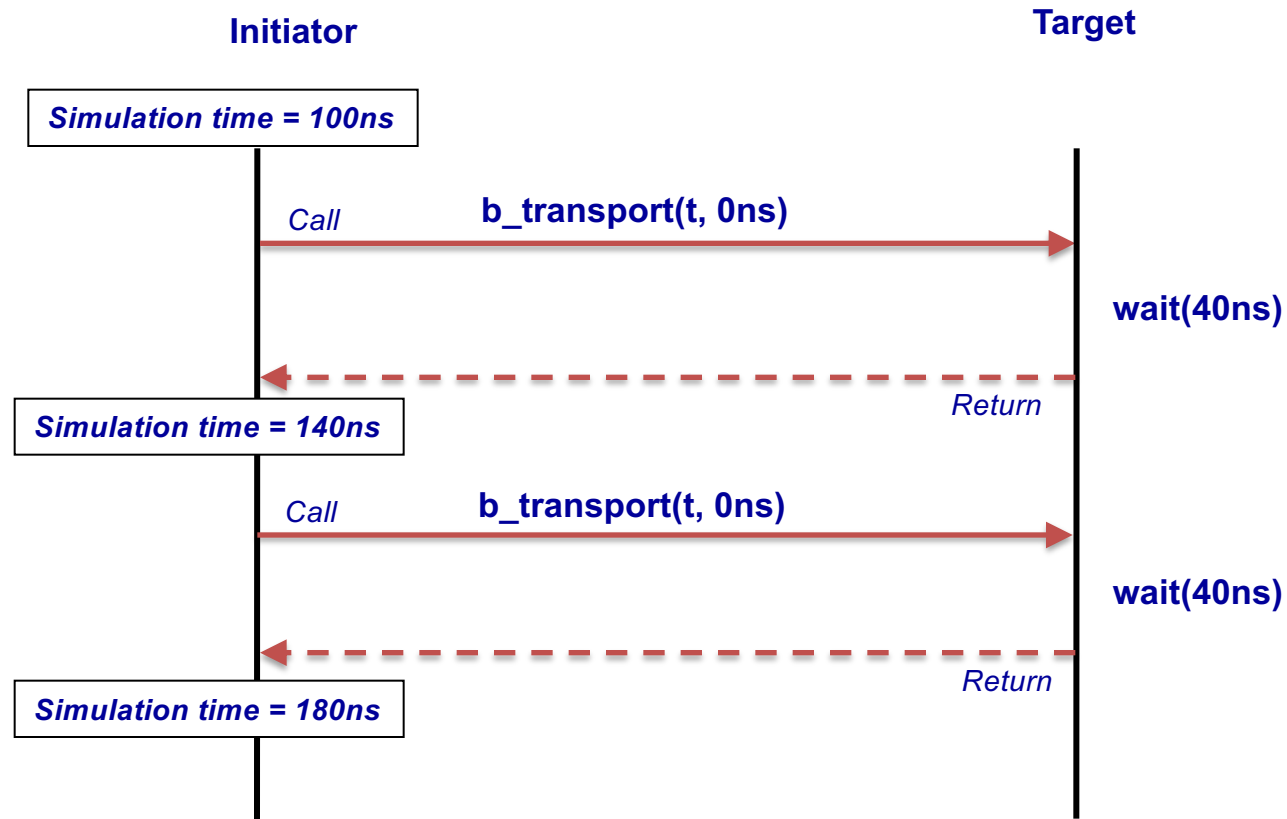
Transaction object



Timing annotation

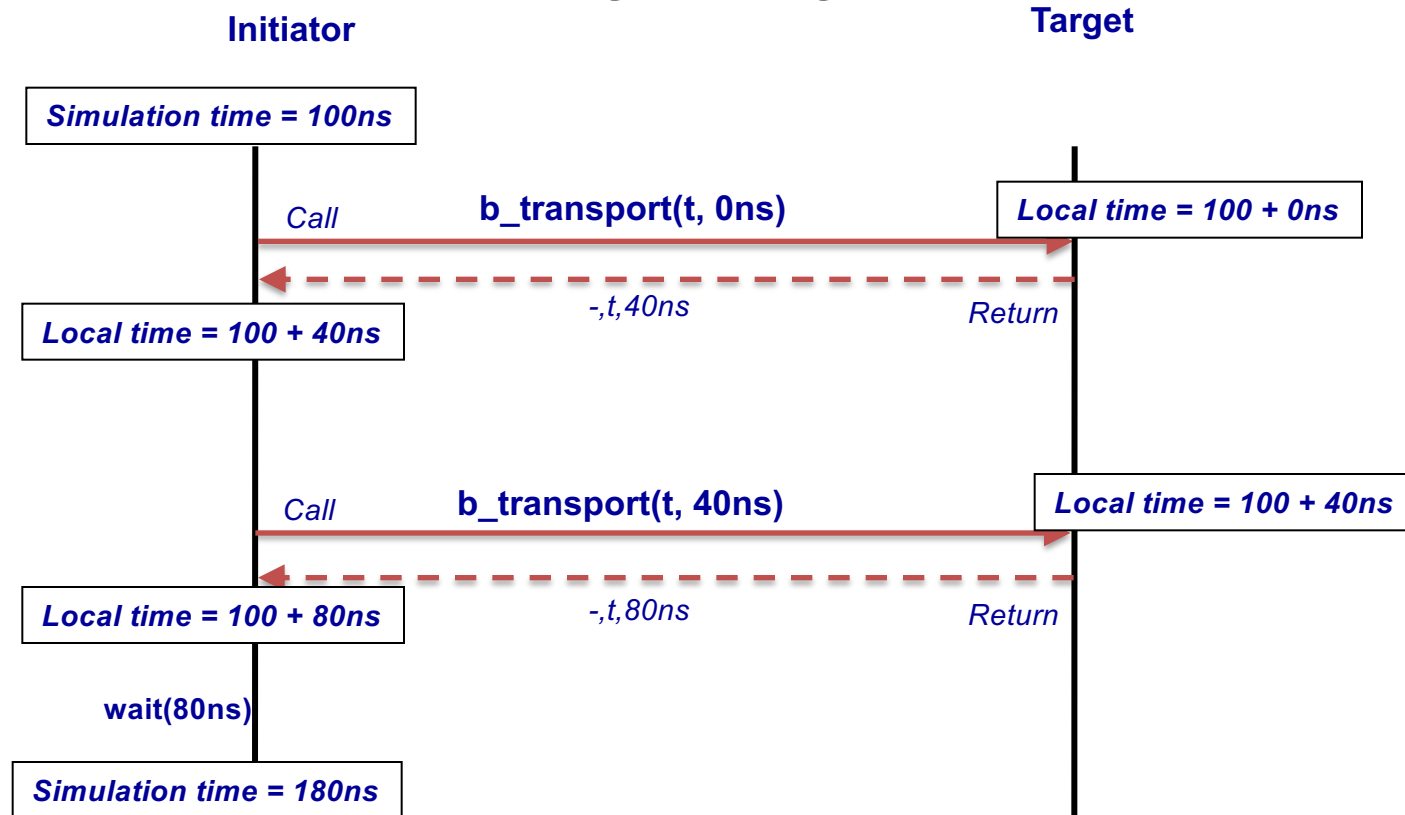
[Link](#)

Blocking Transport – With Wait



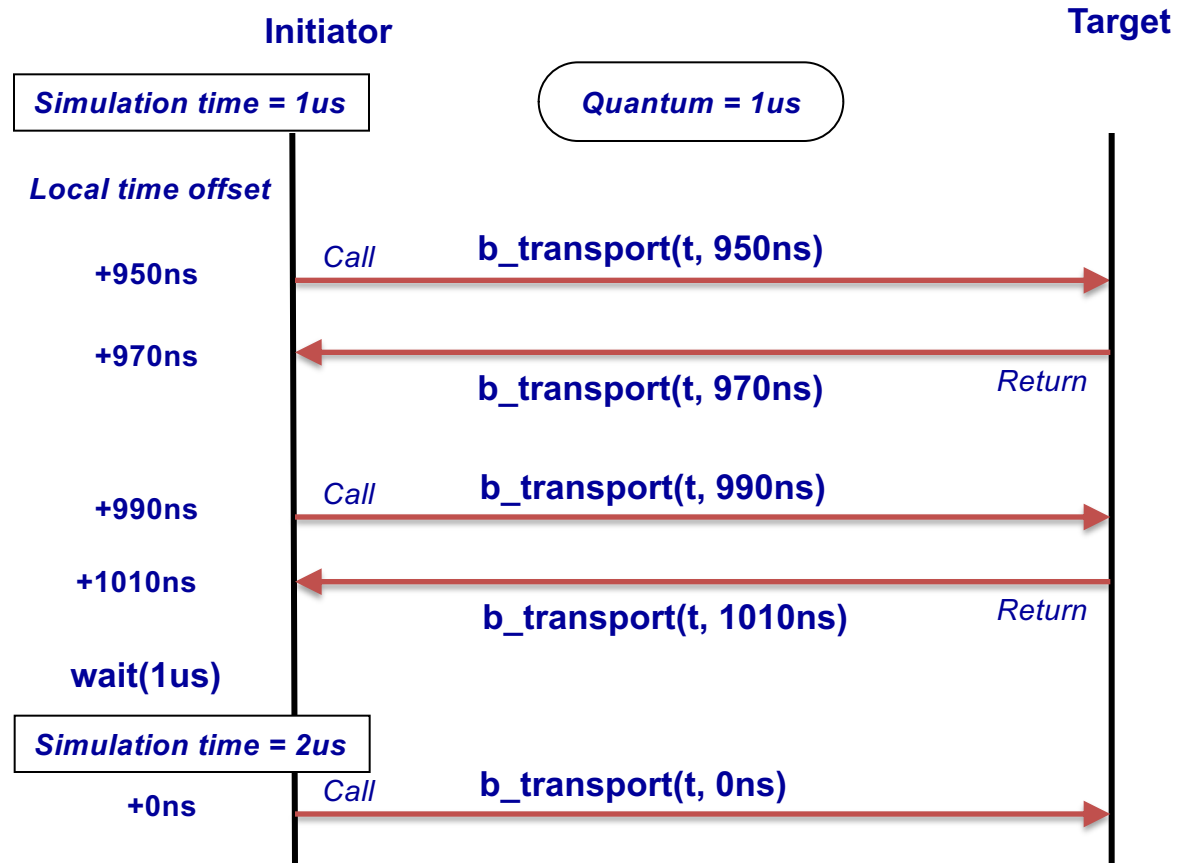
Initiator is blocked until return from `b_transport`

Blocking Transport – With Timing Annotation



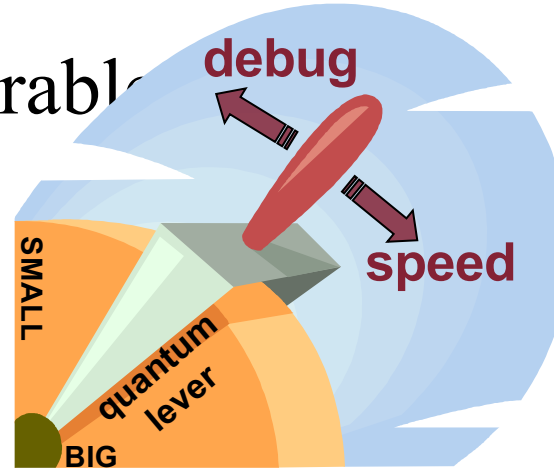
Fewer context switches means higher simulation speed

The Time Quantum



The Quantum Keeper (tlm_quantumkeeper)

- Quantum is user-configurable



- Processes can check local time against quantum

Non-blocking Transport

AT

Non
blocking

```
enum tlm_sync_enum { TLM_ACCEPTED, TLM_UPDATED, TLM_COMPLETED };
```

```
template < typename TRANS = tlm_generic_payload,  
           typename PHASE = tlm_phase>
```

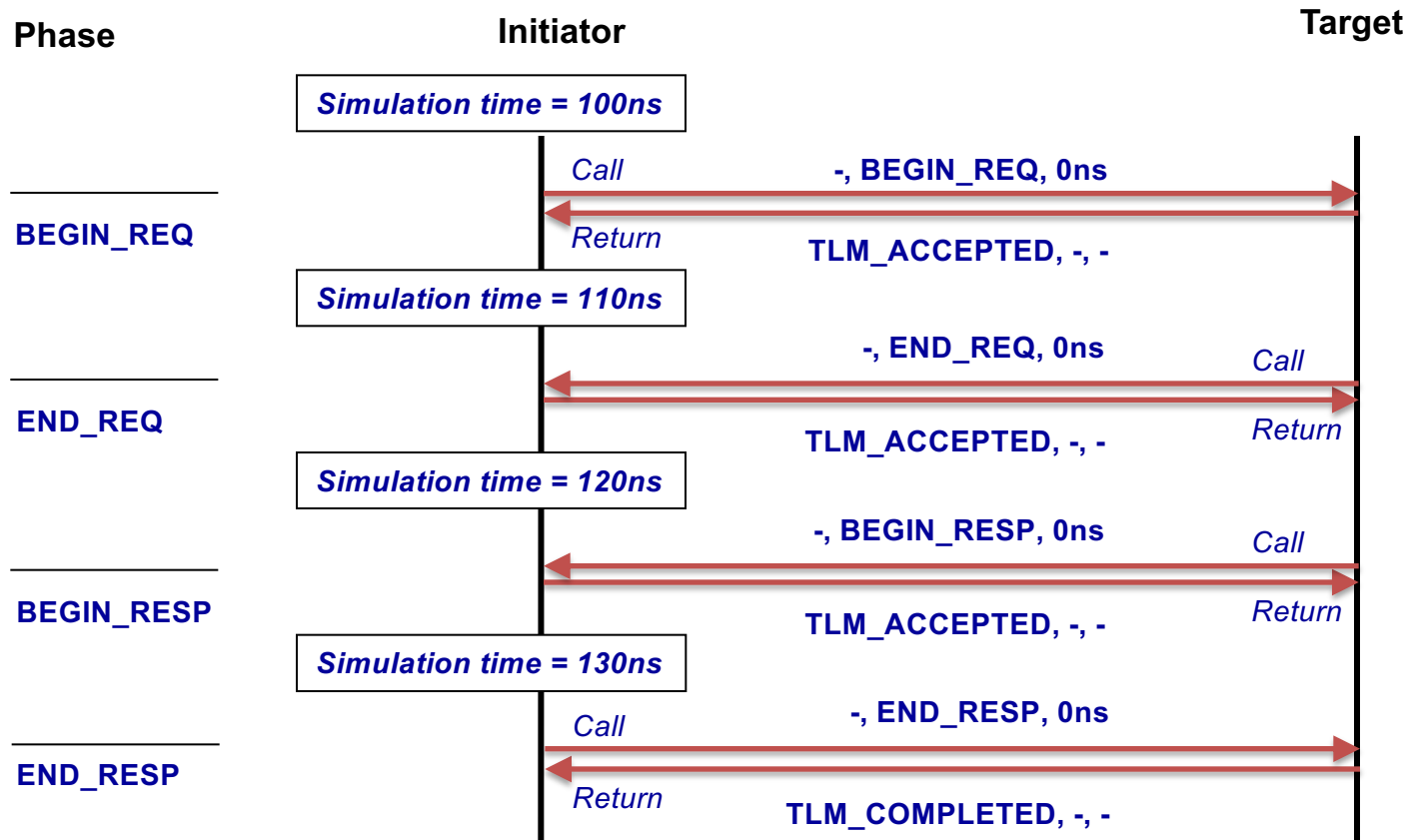
```
class tlm_fw_nonblocking_transport_if : public virtual sc_core::sc_interface {  
public:  
    virtual tlm_sync_enum nb_transport(          TRANS& trans,  
                                           PHASE& phase,  
                                           sc_core::sc_time& t ) = 0;  
};
```

Trans, phase and time arguments set by caller and modified by callee

tlm_sync_enum

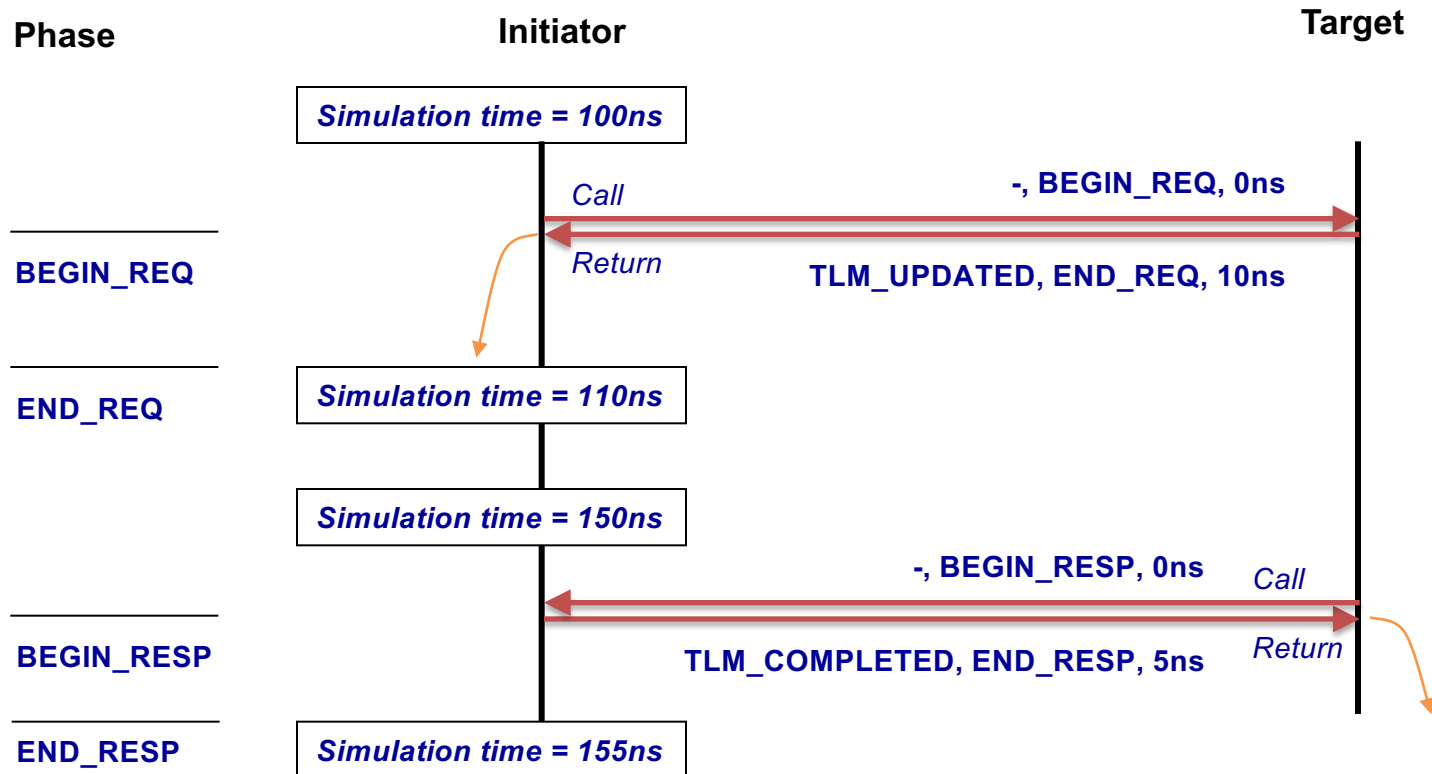
- TLM_ACCEPTED
 - Transaction, phase and timing arguments unmodified (ignored) on return
 - Target may respond later (depending on protocol)
- TLM_UPDATED
 - Transaction, phase and timing arguments updated (used) on return
 - Target has advanced the protocol state machine to the next state
- TLM_COMPLETED
 - Transaction, phase and timing arguments updated (used) on return
 - Target has advanced the protocol state machine [Link](#) straight to the final phase

Using the Backward Path



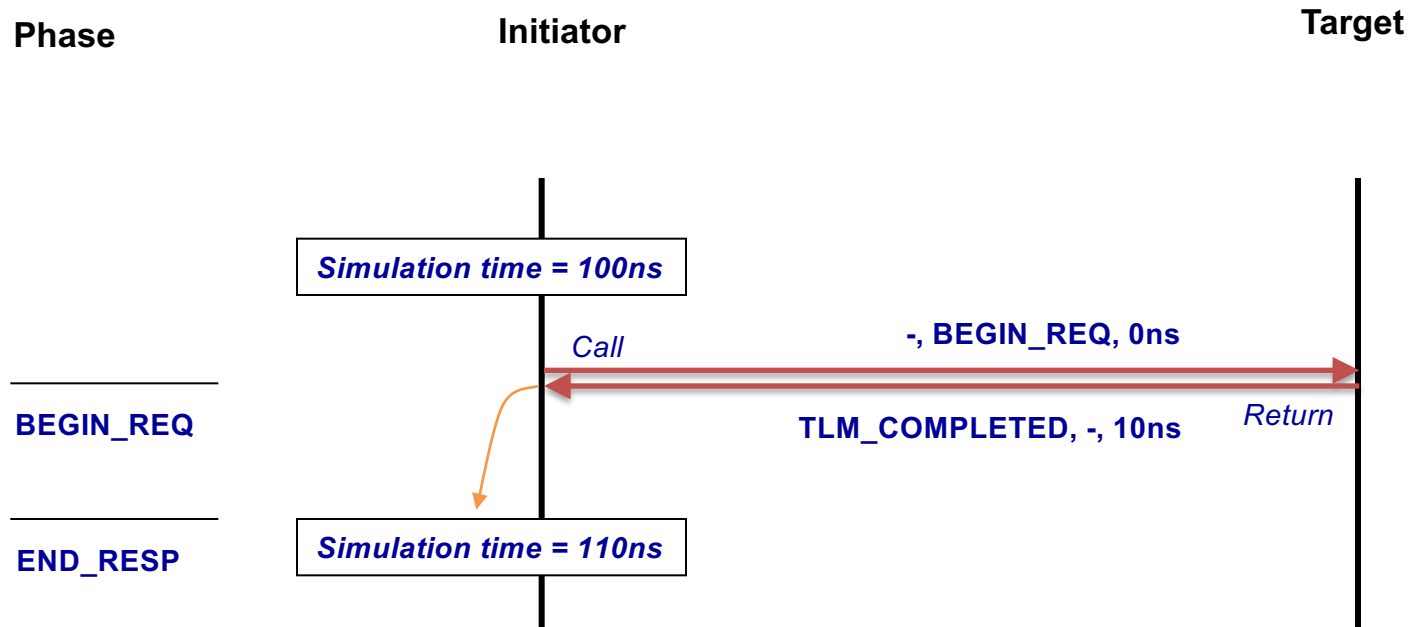
Transaction accepted now, caller asked to wait

Using the Return Path



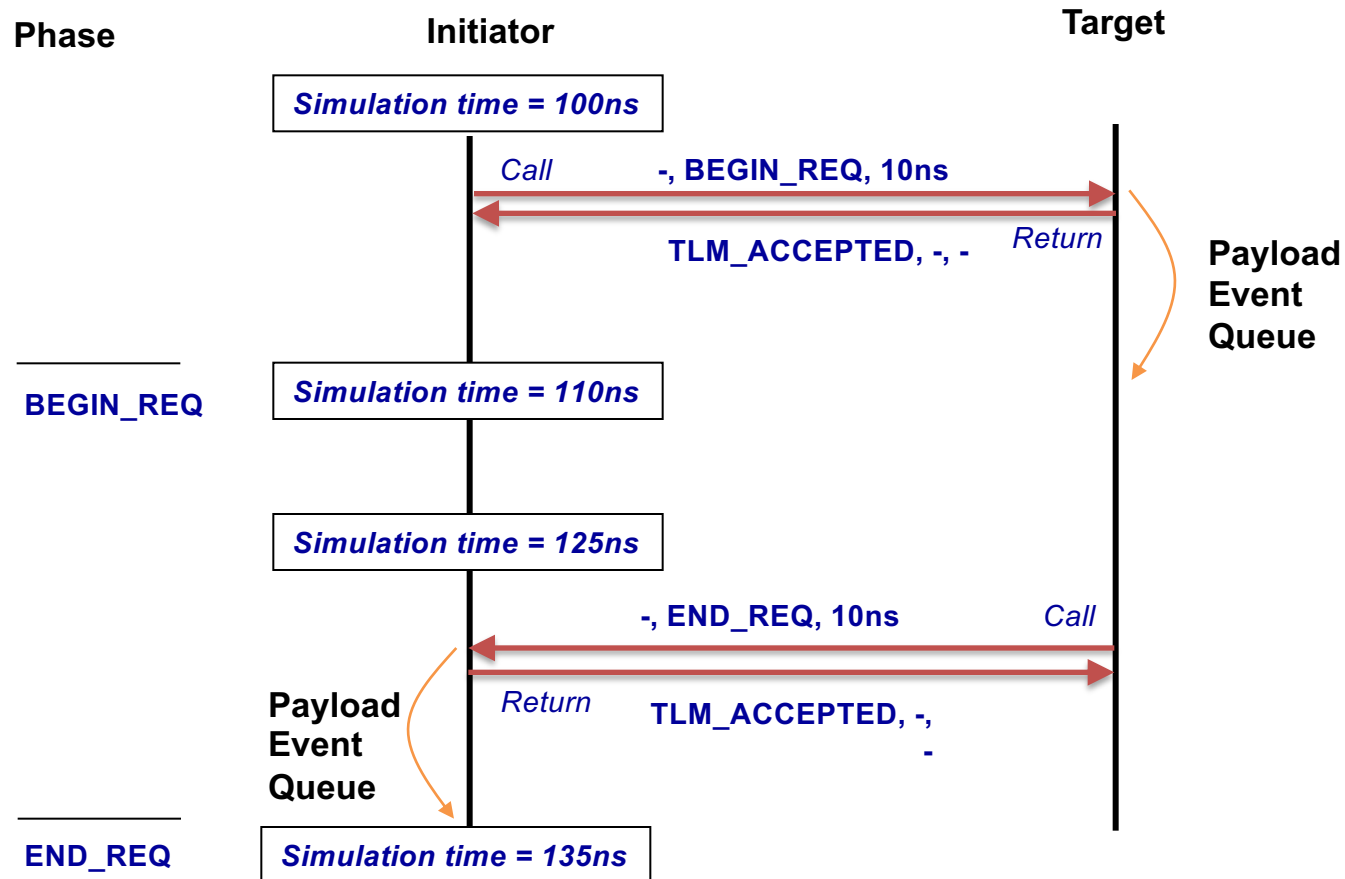
Callee annotates delay to next transition, caller waits

Early Completion



Callee annotates delay to next transition, caller waits

Timing Annotation



DMI and Debug Transport

A blue square icon with the letters 'LT' in white.A red square icon with the letters 'DMI' in white.

- **Direct Memory Interface**

- Gives an initiator a direct pointer to memory in a target, e.g an ISS
- By-passes the sockets and transport calls
- Read or write access by default
- Extensions may permit other kinds of access, e.g. security mode
- Target responsible for invalidating pointer

- **Debug Transport Interface**

- Gives an initiator debug access to memory in a target
- Delay-free
- Side-effect-free

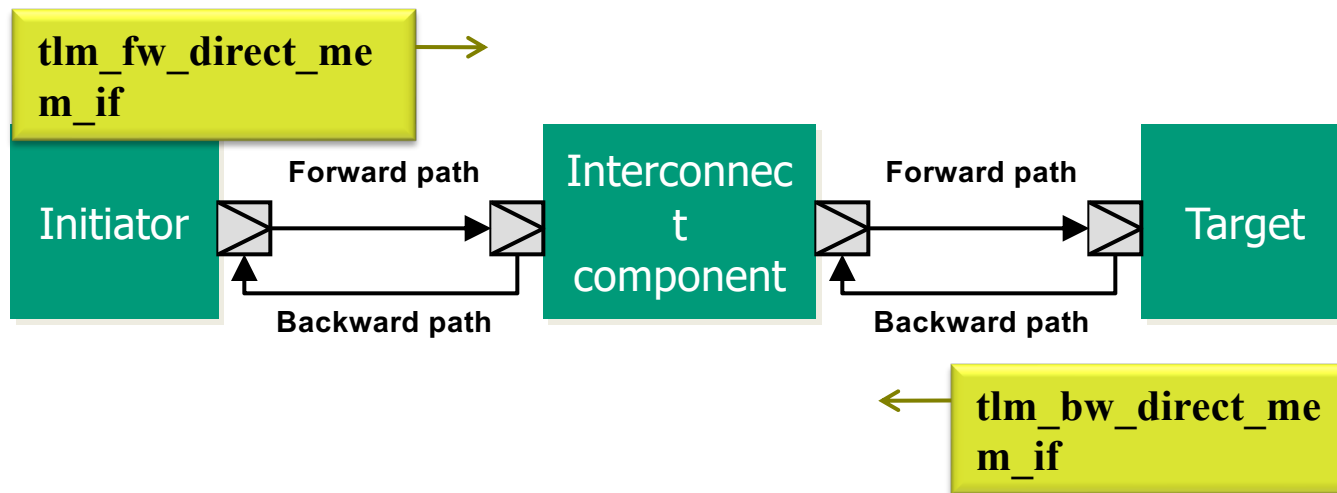
- **May share transactions with transport interface**

Direct Memory Interface

Access requested *Access granted*

↓ ↓

```
status = get_direct_mem_ptr( transaction, dmi_data );
```



```
invalidate_direct_mem_ptr( start_range, end_range );
```

Transport, DMI and debug may all use the generic payload

Interconnect may modify address and invalidated range

DMI Transaction and DMI Data

DMI Transaction

Requests read or write access
For a given address
Permits extensions

class tlm_dmi

```
unsigned char* dmi_ptr  
uint64         dmi_start_address  
uint64         dmi_end_address  
dmi_type_e     dmi_type;  
sc_time        read_latency  
sc_time        write_latency
```

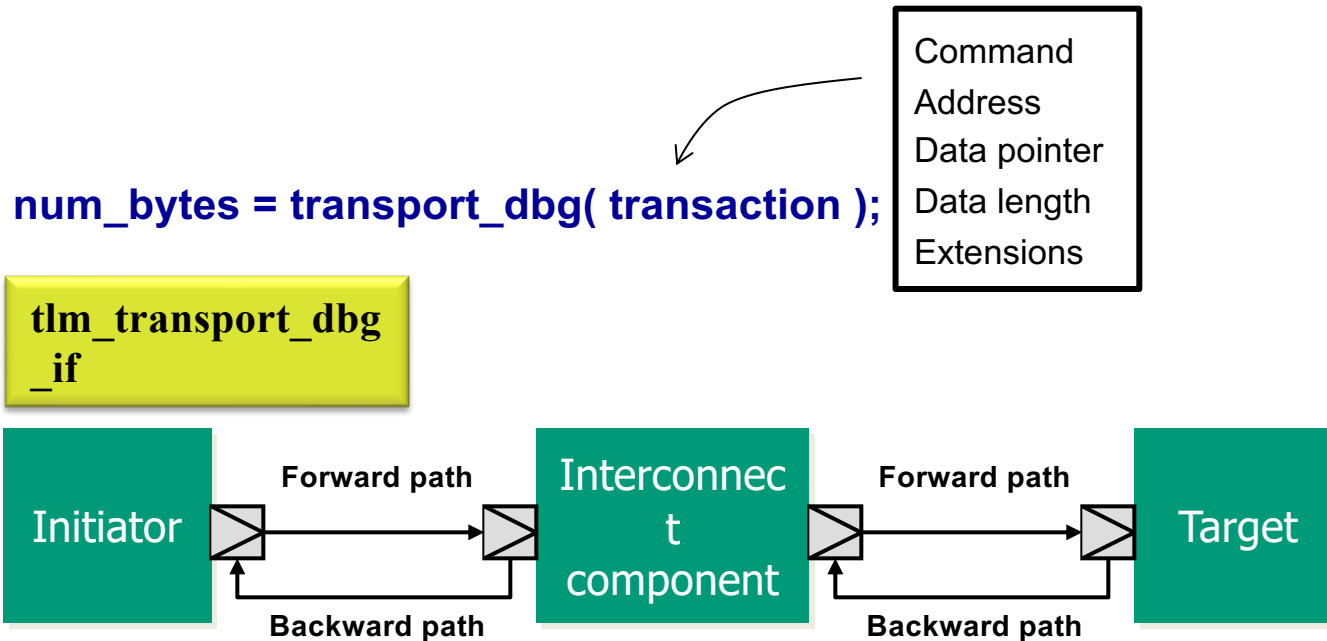
Direct memory pointer

Region granted for given access type

Read, write or read/write

Latencies to be observed by initiator

Debug Transport Interface



Uses forward path only

Interconnect may modify address, target reads or writes data

The Generic Payload

LT

AT

Generic
Payload

- Typical attributes of memory-mapped busses
 - command, address, data, byte enables, single word transfers, burst transfers, streaming, response status
- Off-the-shelf general purpose payload

Specific protocols can use the same generic payload machinery

 - for abstract bus modeling
 - *ignorable* extensions allow full

Generic Payload Attributes

Attribute	Type	Modifiable?	
Command	tlm_command	No	
Address	uint64	Interconnect only	
Data pointer	unsigned char*	No (array – yes)	<i>Array owned by initiator</i>
Data length	unsigned int	No	
Byte enable pointer	unsigned char*	No (array – yes)	<i>Array owned by initiator</i>
Byte enable length	unsigned int	No	
Streaming width	unsigned int	No	
DMI hint	bool	Yes	<i>Try DMI !</i>
Response status	tlm_response_status	Target only	
Extensions	(tlm_extension_base*)[]	Yes	<i>Consider memory management</i>

class tlm_generic_payload

```
class tlm_generic_payload {  
public:
```

Not a template

```
    // Constructors, memory management
```

```
    tlm_generic_payload () ;
```

```
    tlm_generic_payload(tlm_mm_interface& mm);
```

Construct & set mm

Frees all extensions

```
    virtual ~tlm_generic_payload ();
```

Frees mm'd extensions

```
    void reset();
```

```
    void set_mm(tlm_mm_interface* mm);
```

mm is optional

```
    bool has_mm();
```

Incr reference count

```
    void acquire();
```

Decr reference count, 0 => free trans

```
    void release();
```

```
    int get_ref_count();
```

```
    void deep_copy_into(tlm_generic_payload& other) const;
```

```
    ...
```

```
};
```

Memory Management Rules

- **b_transport** – memory managed by initiator, or reference counting (set_mm)
- **nb_transport** – reference counting only
 - Reference counting requires heap allocation
 - Transaction automatically freed when reference count == 0
 - free() can be overridden in memory manager for transactions
 - free() can be overridden for extensions
- When **b_transport** calls **nb_transport**, must add reference counting
 - Can only return when reference count == 0
- **b_transport** can check for reference counting, or assume it could be present

Command, Address and Data

```
enum tlm_command {  
    TLM_READ_COMMAND,  
    TLM_WRITE_COMMAND,  
    TLM_IGNORE_COMMAND  
};
```

Copy from target to data array

Copy from data array to target

Neither, but may use extensions

```
tlm_command    get_command() const ;  
void           set_command( const tlm_command command ) ;
```

```
sc_dt::uint64  get_address() const;  
void           set_address( const sc_dt::uint64 address );
```

```
unsigned char* get_data_ptr() const;  
void           set_data_ptr( unsigned char* data );
```

Data array owned by initiator

```
unsigned int    get_data_length() const;  
void           set_data_length( const unsigned int length );
```

Number of bytes in data array

Response Status

enum tlm_response_status	Meaning
TLM_OK_RESPONSE	Successful
TLM_INCOMPLETE_RESPONSE	Transaction not delivered to target. (Default)
TLM_ADDRESS_ERROR_RESPONSE	Unable to act on address
TLM_COMMAND_ERROR_RESPONSE	Unable to execute command
TLM_BURST_ERROR_RESPONSE	Unable to act on data length or streaming width
TLM_BYTE_ENABLE_ERROR_RESPONSE	Unable to act on byte enable
TLM_GENERIC_ERROR_RESPONSE	Any other error

The Standard Error Response

- A target shall either
 - Execute the command and set
TLM_OK_RESPONSE
 - Set the response status attribute to an error
response
 - Call the SystemC report handler and set
TLM_OK_RESPONSE

Generic Payload Example 1

```
void thread_process() { // The initiator
    tlm::tlm_generic_payload trans;
    sc_time delay = SC_ZERO_TIME;
```

Would usually pool transactions

```
    trans.set_command( tlm::TLM_WRITE_COMMAND );
    trans.set_data_length( 4 );
    trans.set_byte_enable_ptr( 0 );
    trans.set_streaming_width( 4 );
```

```
    for ( int i = 0; i < RUN_LENGTH; i += 4 ) {
        int word = i;
        trans.set_address( i );
        trans.set_data_ptr( (unsigned char*)( &word ) );
        trans.set_response_status( tlm::TLM_INCOMPLETE_RESPONSE );
```

```
        init_socket->b_transport( trans, delay );
```

```
        if ( trans.get_response_status() <= 0 )
            SC_REPORT_ERROR("TLM2", trans.get_response_string().c_str());
```

```
        ...
```

```
    }
```

Generic Payload Extension Methods

- Generic payload has an array-of-pointers to extensions
- One pointer per extension type
- Every transaction can potentially carry every extension type
- Flexible mechanism

template <typename T> T* **set_extension** (T* ext); *Sticky extn*

template <typename T> T* **set_auto_extension** (T* ext); *Freed by ref counting*

template <typename T> T* **get_extension**() const;

template <typename T> void **clear_extension** (); *Clears pointer, not extn object*

template <typename T> void **release_extension** ();
*mm => convert to auto
no mm => free extn object*

Extension Example

```
struct my_extension : tlm_extension<my_extension>
```

User-defined extension

```
{  
    my_extension() : id(0) {}  
    tlm_extension_base* clone() const { ... }  
    virtual void copy_from(tlm_extension_base const &ext) { ... }  
    int id;  
};  
...
```

Pure virtual methods

```
tlm_generic_payload* trans = new tlm_generic_payload( mem_mgr );
```

Heap allocation

```
trans->acquire();
```

Reference counting

```
my_extension* ext = new my_extension;  
ext->id = 1;  
trans.set_extension( ext );
```

```
socket->nb_transport_fw( *trans, phase, delay );  
trans.release_extension<my_extension>();
```

Freed when ref count = 0

```
trans->release();
```

Trans and extn freed

Base Protocol - Coding Styles

- Loosely-timed is typically
 - Blocking transport interface, forward and return path
 - 2 timing points
 - Temporal decoupling and the quantum keeper
 - Direct memory interface
- Approximately-timed is typically
 - Non-blocking transport interface, forward and backw
 - 4 phases
 - Payload event queues

Base Protocol and tlm_phase

- The base protocol = tlm_generic_payload + tlm_phase
- tlm_phase has 4 phases, but can be extended to add new phases

```
enum tlm_phase_enum { UNINITIALIZED_PHASE = 0,  
                      BEGIN_REQ=1, END_REQ, BEGIN_RESP, END_RESP };
```

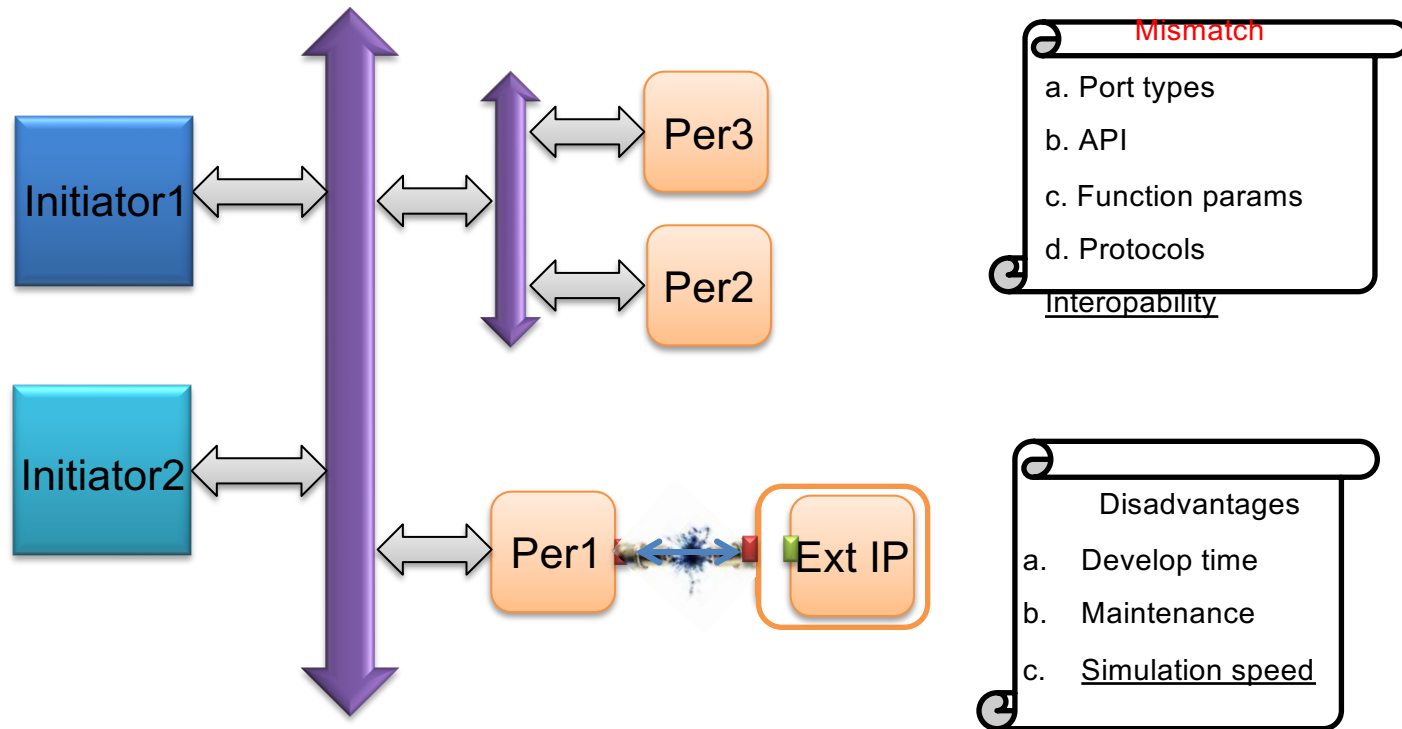
```
class tlm_phase {  
public:  
    tlm_phase();  
    tlm_phase( unsigned int id );  
    tlm_phase( const tlm_phase_enum& standard );  
    tlm_phase& operator= ( const tlm_phase_enum& standard );  
    operator unsigned int() const;  
};
```

```
#define DECLARE_EXTENDED_PHASE(name_arg) \  
class tlm_phase_##name_arg : public tlm::tlm_phase { \  
...  
}
```

Base Protocol Rules

- Base protocol phases
 - BEGIN_REQ → END_REQ → BEGIN_RESP → E
 - Must occur in non-decreasing simulation time order
 - Only permitted one outstanding request or response p
 - Phase must change with each call (other than ignorab
 - May complete early
- Generic payload memory management rules
- Extensions must be ignorable
- Target is obliged to handle mixed b_transport / nb_transport
- Write response must come from target

Why Do We Need TLM Standard ?



1
3
2

[Link](#)

Acronyms

- Initiator – module that initiates new transactions
- Target – module that responds to transactions initiated by initiator
- Transaction – data structure (C++ object) passed between initiators and targets using function calls

[Link](#)

Code Snippet - Example 1

1/4

```
#include "tlm.h" → TLM 2.0 interoperability standard  
#include "tlm_utils/simple_initiator_socket.h"      TLM Utility  
#include "tlm_utils/simple_target_socket.h"
```

```
struct simple_initiator : sc_module {  
    tlm_utils::simple_initiator_socket<simple_initiator>  
    init_socket;  
    SC_CTOR(simple_initiator) : init_socket("init_socket") { ...}  
};  
struct memory : public sc_module {  
    tlm_utils::simple_target_socket<memory> mem_socket;  
    SC_CTOR(memory):mem_socket("mem_socket") { ...}  
};
```

1
3
4

Code Snippet - Example 1

2/4

```
struct Top : sc_module {
    simple_initiator *initiator;
    memory *mem;
    SC_CTOR(Top) {
        initiator = new
simple_initiator("simple_initiator");
        mem = new memory("memory");
        initiator->init_socket.bind(mem->mem_socket);
    }
};

int sc_main() {
    Top top("top");
    sc_start();
    return 0;
}
```

1
3
5

Code Snippet - Example 1

3/4

```
struct simple_initiator : sc_module {
    tlm_utils::simple_initiator_socket<simple_initiator>
init_socket;
    SC_CTOR(simple_initiator) : init_socket("init_socket") {
        SC_THREAD(test_process);
    }
    void test_process() {
        while(1) {
            tlm::tlm_generic_payload *gp = new
tlm::tlm_generic_payload;
            sc_time delay;
            // Set different fields of GP
            init_socket->b_transport(*gp, delay);
            ....
        }
    }
}
```

1
3
6

Code Snippet - Example 1

4/4

```
struct memory : sc_module {
    tlm_utils::simple_target_socket<memory> mem_socket;
    SC_CTOR(memory):mem_socket("mem_socket") {
        ...
        mem_socket.register_b_transport(this,
&memory::b_transport);
    }
    void b_transport(tlm::tlm_generic_payload &gp, sc_time
&delay) { ...}
};
```

[Link](#)

1
3
7

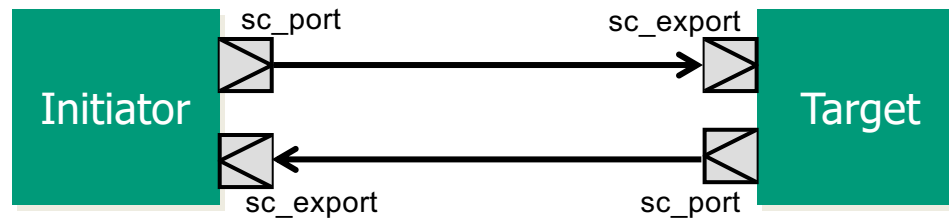
Sockets

LT

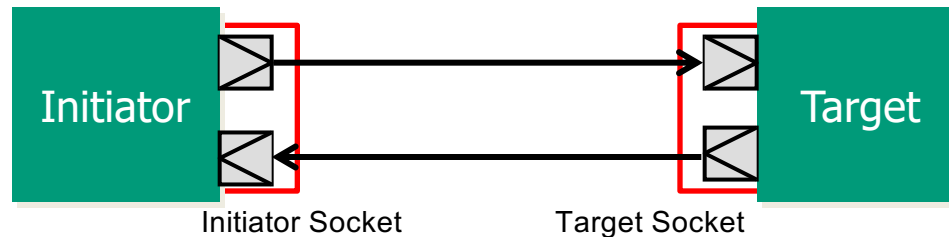
AT

Sockets

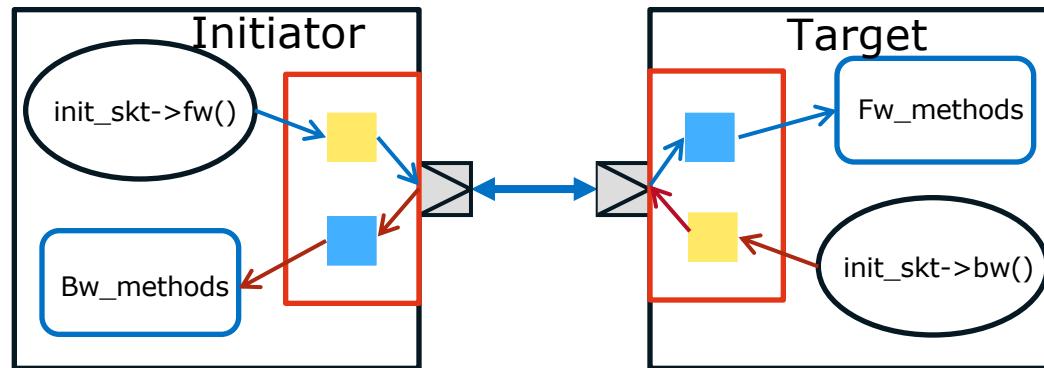
- Transaction involve communication in both direction



- Initiator and Target Sockets makes it convenient



Sockets



- Implemented as SystemC channels

```
struct Initiator : sc_module, tlm::tlm_bw_transport_if<>
{}
```

```
struct Target : sc_module, tlm::tlm_fw_transport_if<> {}
```

Sockets

- Example Initiator module

```
struct initiator : sc_module, tlm::tlm_bw_transport_if<> {  
    tlm::tlm_initiator_socket<> init_socket;  
  
    ...  
  
    // Must implement all backward calls  
    tlm::tlm_sync_enum nb_transport_bw(...) {}  
    void invalidate_direct_mem_ptr(...) {}  
};
```

1
4
0

Sockets

- Example Target module

```
struct target : sc_module, tlm::tlm_fw_transport_if<> {  
    tlm::tlm_target_socket<> target_socket;  
  
    ...  
  
    // Must implement all forward calls  
    void b_transport(...) {}  
  
    tlm::tlm_sync_enum nb_transport_fw(...) { }  
  
    bool get_direct_mem_ptr(...) {}  
  
    unsigned int transport_dbg(...) { }  
  
};
```

[Link](#)

1
4
1

LT Mechanisms for Increasing Simulation Speed

- Impediments to simulation speed
 - Context switches
 - Reduce context switches with temporal decoupling
 - Function call hierarchy
 - By-pass bus with direct access to memory (DMI)

[Link](#)

Phases

- BEGIN_REQ
 - Initiator acquires the bus
 - Connection becomes busy and clocks further requests
- END_REQ
 - Target 'accepts' the request
 - Bus free to start additional requests
- BEGIN_RESP
 - Target acquires the bus to provide response
 - Bus becomes 'busy'
- END_RESP
 - Initiator acknowledges response
 - Bus is freed
 - Payload reference freed-up

[Link](#)

Quantum Keeper Example



```
struct Initiator: sc_module  
{
```

```
    tlm_utils::simple_initiator_socket<Initiator> init_socket;  
    tlm_utils::tlm_quantumkeeper m_qk;
```

The quantum keeper

```
    SC_CTOR(Initiator) : init_socket("init_socket") {
```

```
        ...
```

```
        m_qk.set_global_quantum( sc_time(1, SC_US) );  
        m_qk.reset();
```

Replace the global quantum

Recalculate the local quantum

```
    }
```

```
    void thread() { ...
```

```
        for (int i = 0; i < RUN_LENGTH; i += 4) {
```

```
            ...
```

```
            delay = m_qk.get_local_time();
```

```
            init_socket->b_transport( trans, delay );
```

```
            m_qk.set( delay );
```

```
            m_qk.inc( sc_time(100, SC_NS) );
```

```
            if ( m_qk.need_sync() )
```

```
                m_qk.sync();
```

Time consumed by transport

Further time consumed by initiator

*Check local time against quantum
and sync if necessary*

```
        }
```

```
    }
```

```
};
```

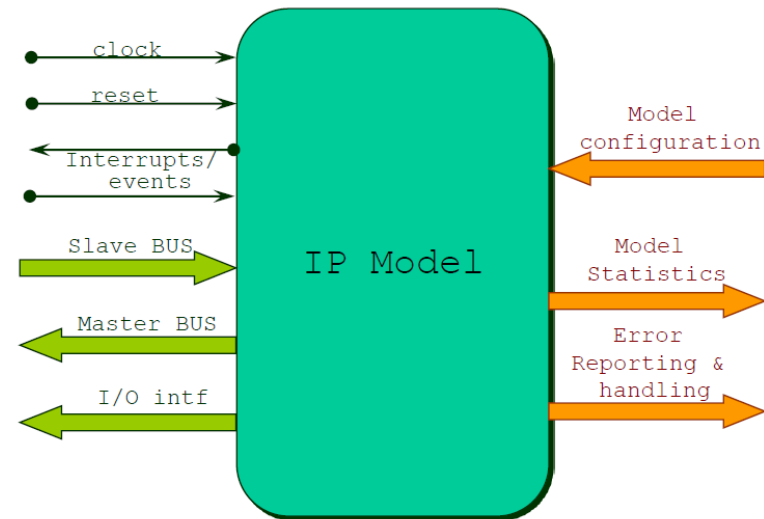

Challenges ahead (1/3)



- Need for more performance!
 - TLM2.0 LT style modeling is a very good step forward.
 - Simulations with speed from tens to hundreds of MIPS achievable, with LT style modeling and temporal decoupling
 - But . . . Lot more space to cover!
 - Multi-core simulation (16 cores and more)
 - Board-level simulation (Multiple SoC simulation)
 - Options
 - Parallel SystemC
 - Parallel Discrete Event Simulation
 - Had a bit of success in TCIC6618 simulator, with board-level simulation
 - Native support on SystemC PoC?

Challenges ahead (2/3)

- Model-Model Interoperability standards
 - TLM 2.0 interface standard increased the interoperability
 - More space to cover, yet!
 - Non-memory mapped interface standardization
 - Interrupts, serial interfaces
 - I/O interface standardization
 - PCIe, UART, I2C, SRIO, etc.,.
- Tool-Model Interoperability standards
 - Aware of OSCI initiative on CCI standards for configuration and control
 - Current gaps
 - Software Debugger and ISA model interoperability standards
 - Use-case description for rapid-prototyping and architecture-studies



Challenges ahead (3/3)

- OSCI SystemC Proof-of-concept simulator is available as static library.
 - SystemC 2.3 Linux builds are available as shared-objects.
 - Windows builds are on static library.
- SoC/board-level simulation, one does not have to build/link the entire simulation platform, if the solution was available as shared-objects/DLL.
 - Next-gen Wireless infrastructure simulator takes about 2 hours to build on windows.
 - This can be greatly reduced, if IP models are independently built and delivered (for patch updates) as shared-objects
 - The Proof-of-concept simulator implementation need changes to make it a DLL on windows



CHALLENGES AHEAD

Challenges(1/3)

- Need for more performance!
 - TLM 2.0 LT style modeling is a very good step forward
 - Simulations with speed from tens to hundreds of MIPS achievable, with LT style modeling and temporal decoupling
 - But .. Lot more space to cover
 - Multi-core simulation
 - Board level simulation(multiple soc simulation)
 - Options
 - Parallel simulator
 - Pararalled discrete event simulation
 - Had a bit of success in TCIC6618 simulator, with board-level simulation.
 - Native support on systemC poC?

Challenges(2/3)...

- Model-model interoperability standard
 - TLM 2.0 interface standard increased interoperability.
 - More space to cover
 - Non memory mapped interfaces
 - Interrupts
 - Serial interrupts
 - IO interfaces
 - PCIr
 - UART
 - I2C etc.

Challenges(3/3)

- Tool-model interoperability standards
 - Aware of OSCI initiative on CCI standards for configuration and control
 - Current gaps
 - Software debugger and ISA model interop standards.
 - Use-case description for rapid-prototyping and architecture-studies
- Windows DLL
 - Support in 2.3.1 draft version.